

# 现代密码学

MODERN CRYPTOGRAPHY

王剑锋

网络与信息安全学院

<http://web.xidian.edu.cn/jfwang>



## 第二讲 私钥加密技术

- 1 私钥密码体制概述
- 2 流密码
- 3 DES算法
- 4 AES算法
- 5 中国商用密码算法SM4
- 6 分组密码的工作模式

## □ 私钥密码体制

- 私钥密码体制又称为单钥密码体制或对称密码体制，主要特征是加密密钥和解密密钥相同。
- 私钥密码体制的优势：加、解密速度更快，密钥相对较短，可以低成本的软硬件实现。
- **私钥密码体制的主要问题**：安全的密钥管理 (Key Management)，即如何安全高效地产生、分发、保管密钥，尤其是在大规模的网络环境中。
- **私钥密码体制**：
  - 明文消息按字符（如二元数字）逐位地加密，称为流密码；
  - 将明文消息分组（含多个字符），逐组地进行加密，称为分组密码。

## □ 流密码概述

- Shannon 已经证明 “一次一密” 在理论上是不可破译的。
  - 安全、高效。
  - 随机密钥的长度至少要等于明文长度。
- **流密码**（也称**序列密码**） 是一次一密系统的弱化版本。
  - 理论比较成熟，而且实现简单、速度快、错误传播少。
  - 军事和外交等领域的主要密码体制之一。
  - 流密码中使用一个伪随机序列来代替随机的密钥流。因此，不可能真正的实现完善保密。

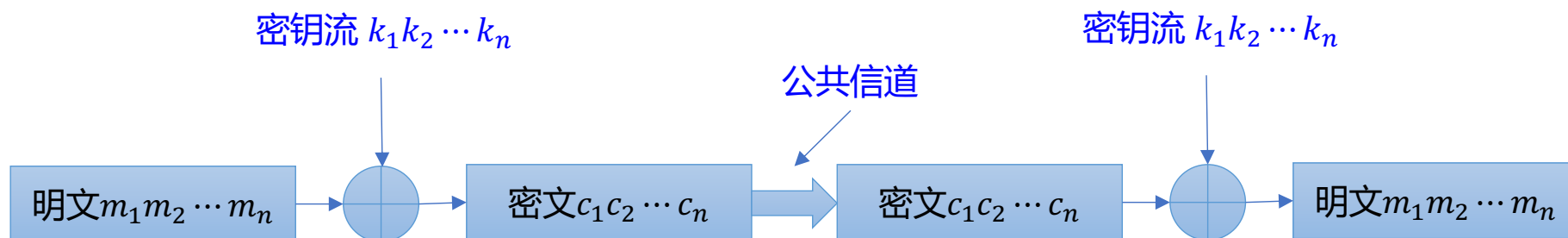
## □ 流密码概述

- 设明文为  $m = m_1 m_2 \cdots m_n$ ，由种子密钥通过伪随机数发生器得到的密钥流为：  $k = k_1 k_2 \cdots k_n$ ，则**加密变换为**：

$$C = c_1 c_2 \cdots c_n$$

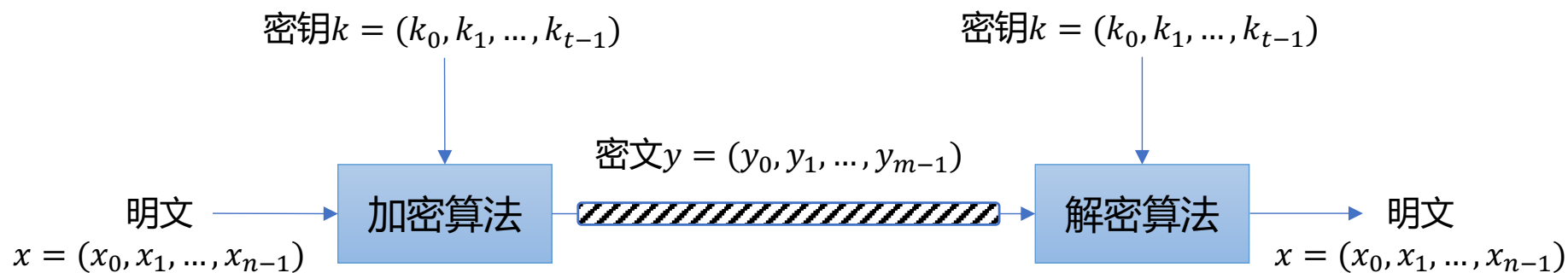
其中  $c_i = m_i \oplus k_i$  ( $i = 1, 2, \dots, n$ )， $\oplus$ 表示模 2 加法（异或）。

- **解密变换为**：  $m_i = c_i \oplus k_i$  ( $i = 1, 2, \dots, n$ )。



## □ 分组密码概述

- **分组密码**是将明文消息编码表示后的数字序列  $x_1, x_2, \dots, x_i, \dots$  划分成长为  $n$  的组  $x = (x_0, x_1, \dots, x_{n-1})$ , 各组分别在密钥  $k = (k_0, k_1, \dots, k_{t-1})$  控制下变换成等长的输出数字序列  $y = (y_0, y_1, \dots, y_{m-1})$  (长为  $m$  的矢量)。一般情况下,  $m = n$ 。
- 分组密码与流密码的不同之处在于输出的每一位数字不是只与相应时刻输入的明文数字有关, 而是与一组长度为  $n$  的明文数字有关。



## □ 分组密码概述

- 分组密码的设计问题在于找到一种算法，能在密钥控制下从一个足够大且足够好的置换子集中，简单而迅速地选出一个置换，用来对当前输入的明文的数字组进行加密变换。设计的算法应满足下述要求：
  - 分组长度要足够大，防止明文穷举攻击奏效。
  - 密钥量要足够大，尽可能消除弱密钥并使所有密钥同等地好，以防止密钥穷举攻击奏效。
  - 由密钥确定置换的算法要足够复杂，充分实现明文与密钥的扩散和混淆，没有简单的关系可循，要能抗击各种已知的攻击。
  - 加密和解密运算简单，易于软件和硬件高速实现。
  - 数据扩展尽可能小。一般无数据扩展，在采用同态置换和随机化加密技术时可引入数据扩展。
  - 差错传播尽可能地小。

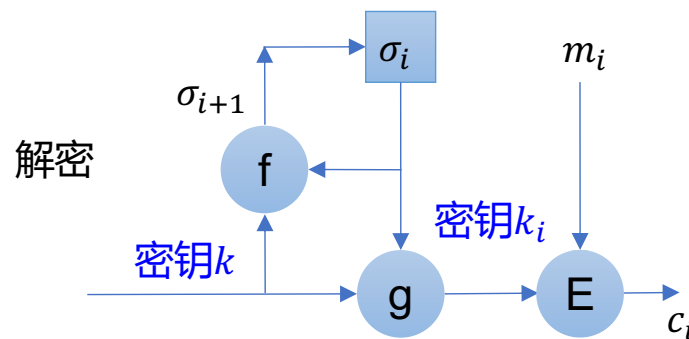
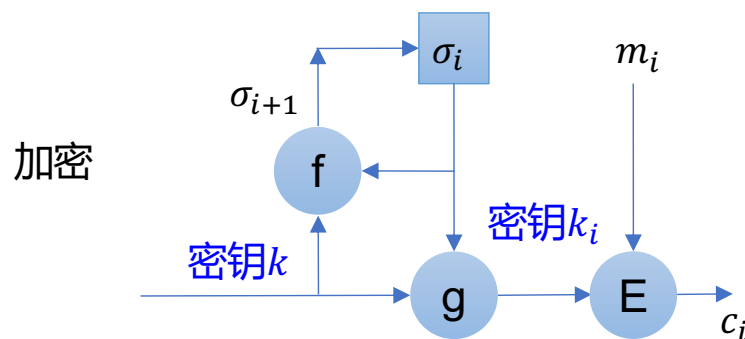
### □ 流密码的基本概念

- 流密码是将明文划分成字符（如单个字母），或其编码的基本单元（如数字0, 1），每一字符分别与密钥流对应字符“作用”，从而进行加密，解密时以同步产生的同样的密钥流实现。
- 流密码强度完全依赖于密钥流产生器所生成序列的随机性（Randomness）和不可预测性（Unpredictability）。
- 因此其核心问题是密钥流生成器的设计。
- 保持收发两端密钥流的精确同步是实现可靠解密的关键。



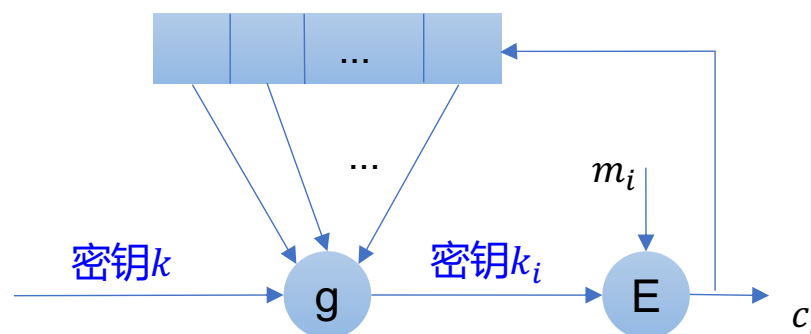
### □ 流密码分类

- **同步流密码**：如果密钥流产生的算法和明文（密文）无关，则所产生的密钥序列也与明文（密文）无关，称这类流密码为同步流密码（Synchronous Stream Cipher）。
  - **无错误传播**：各符号之间真正独立，一个传播错误只影响一个符号，不会影响到后继的符号。
  - **同步**：发送方和接收方必须保持精确的、用同样的密钥并作用在同样的位置上，才能正确地解密。



### □ 流密码分类

- **自同步流密码**：如果密钥流产生算法和明文（密文）相关，则所产生的密钥序列也与明文（密文）相关，称这类流密码为自同步流密码（Self-Synchronous Stream Cipher）。
  - **有限错误传播**：设密钥序列产生器具有 $n$ 位存储，则一个符号的传播错误只影响到后面 $n$ 符号的解密。
  - **自同步**：只要接收方连续收到 $n$ 个正确的密文符号，密钥序列产生器便会自动地恢复同步。
  - **消除明文统计特性**。



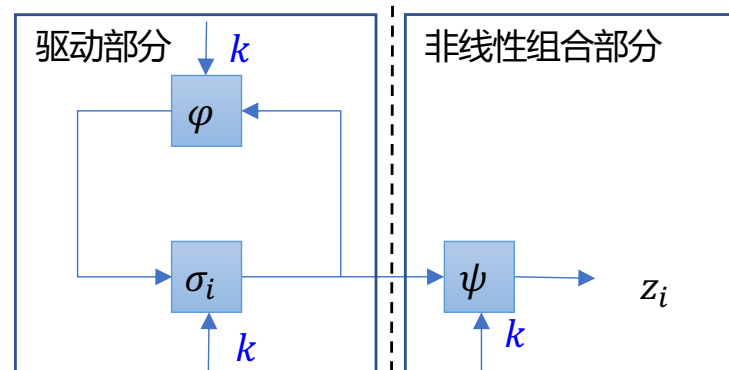
### □ 密钥流的要求

- **极大的周期**：随机序列是非周期的，而按**任何算法**产生的序列都是**周期**的，因此应要求密钥流具有尽可能大的周期。
- **良好的统计特性**：随机序列有**均匀**的**游程分布**
  - 游程指序列中相同符号的连续段，其前后均为异种符号  
 $\dots\dots 0\ 111\ 0000\ 10\dots\dots$
  - 有长为3的1游程、长为4的0游程、长为1的1游程。一般要求其在周期内满足：**同样长度**的**0游程**和**1游程**的个数相等，或近似相等
- **很高的线性复杂度**：不能用级数较小的线性移位寄存器LFSR近似代替。
- 用统计方法由密钥序列  $k_0 k_1 k_2 \cdots k_i \cdots$  提取密钥生成器结构或种子密钥在计算上不可行。

### □ 密钥流生成器的组成

- **驱动部分**：控制生成器的状态序列，为非线性组合部分提供统计性能良好的序列。

- 周期很大
- 分布较随机



- **非线性部分**：将驱动部分提供的序列组合成密码特性好的序列。
  - 可隐蔽驱动序列与密钥 $k$ 之间明显的依赖关系。

目前密钥流生成器大都基于**移位寄存器FSR**

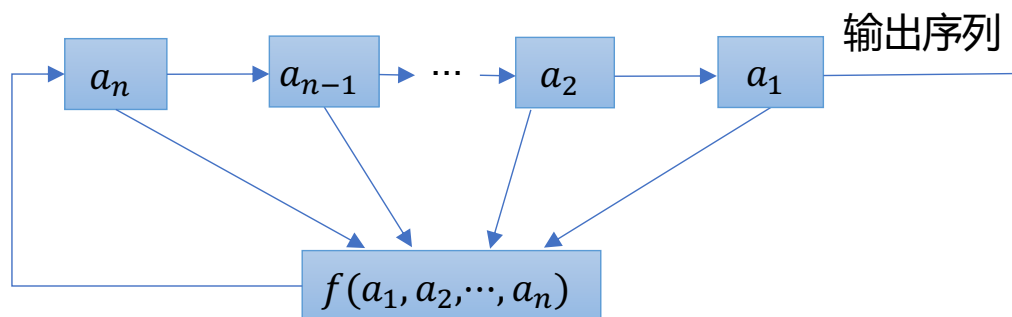
- 基于移位寄存器的密钥流序列称为**移位寄存器序列**。

### □ 密钥流生成器的组成

- 通常由线性移位寄存器 (LFSR) 和一个非线性组合函数即布尔函数组合, 构成一个密钥流生成器。
  - 由一个线性移位寄存器和一个滤波器构成
  - 由多个线性移位寄存器和一个组合器构成
- 产生密钥流最重要的部件是线性反馈移位寄存器 (LFSR, Linear Feedback Shift Register), 主要基于以下原因:
  - 能产生周期大、统计特性好的序列。
  - 非常适合硬件实现。
  - 能够应用代数方法进行很好的分析。

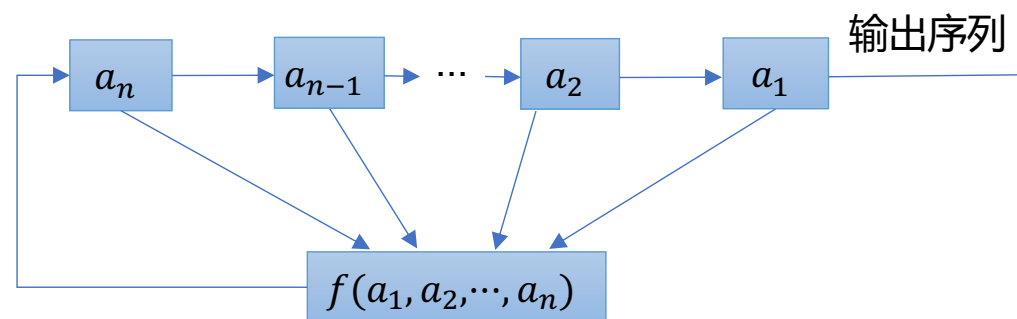
### □ 反馈移位寄存器

- GF(2)上一个n级反馈移位寄存器由n个二元存储器与一个反馈函数  $f(a_1, a_2, \dots, a_n)$  组成。
  - 每个存储器称为移位寄存器的一级。
  - 在任一时刻，这些级的内容构成该FSR的状态；对应于一个GF(2)上的n维向量，共有  $2^n$  种可能的状态。
  - 每一时刻的状态可用 n 维向量  $(a_1, a_2, \dots, a_n)$  表示，其中  $a_i$  是第 i 级存储器的内容。



### □ 反馈移位寄存器

- 初始状态由用户确定
- 反馈函数 $f(a_1, a_2, \dots, a_n)$ 是  $n$  元布尔函数，即函数的自变量和因变量只取 0 和 1 这两个可能的值。函数中的运算有逻辑与、逻辑或、逻辑补等运算。



- **例** 如图是一个3级反馈移位寄存器，其初始状态为 $(a_1, a_2, a_3) = (1, 0, 1)$ ，输出见右下表。

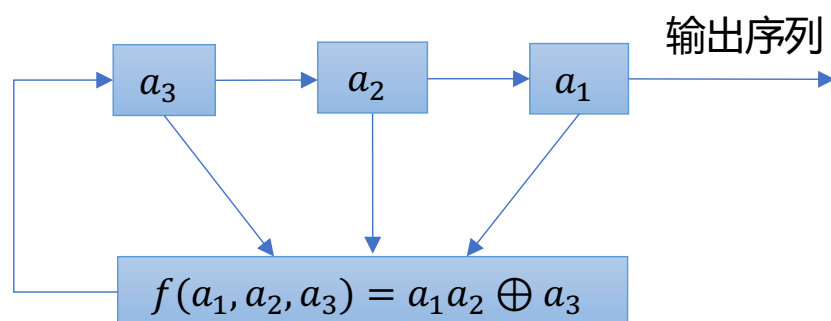


图 一个3级反馈移位寄存器

即输出序列为101110111011..., 周期为4。

| 状态<br>$(a_1, a_2, a_3)$ | 输出 |
|-------------------------|----|
| 1 0 1                   | 1  |
| 1 1 0                   | 0  |
| 1 1 1                   | 1  |
| 0 1 1                   | 1  |
| 1 0 1                   | 1  |
| 1 1 0                   | 0  |

表 一个3级反馈移位寄存器的  
状态和输出

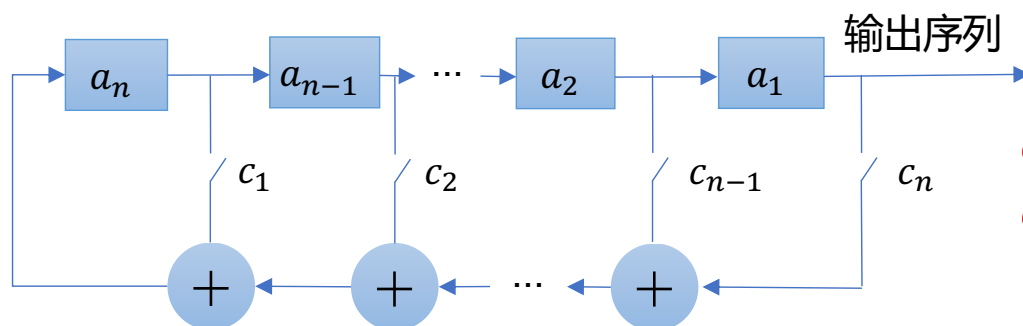


### □ 线性反馈移位寄存器LFSR (linear feedback shift register)

- 如果反馈函数 $f(a_1, a_2, \dots, a_n)$ 是 $a_1, a_2, \dots, a_n$ 的线性函数, 则称为线性反馈移位寄存器 (LFSR), 反馈函数  $f$  可写为:

$$f(a_1, a_2, \dots, a_n) = c_n a_1 \oplus c_{n-1} a_2 \oplus \dots \oplus c_1 a_n$$

- 输出序列 $\{a_t\}$ 满足:  $a_{n+t} = c_n a_t \oplus c_{n-1} a_{t+1} \oplus \dots \oplus c_1 a_{n+t}, t = 1, 2, \dots$
- 线性反馈移位寄存器: 实现简单、速度快、有较为成熟的理论, 成为构造密钥流生成器的最重要的部件之一。

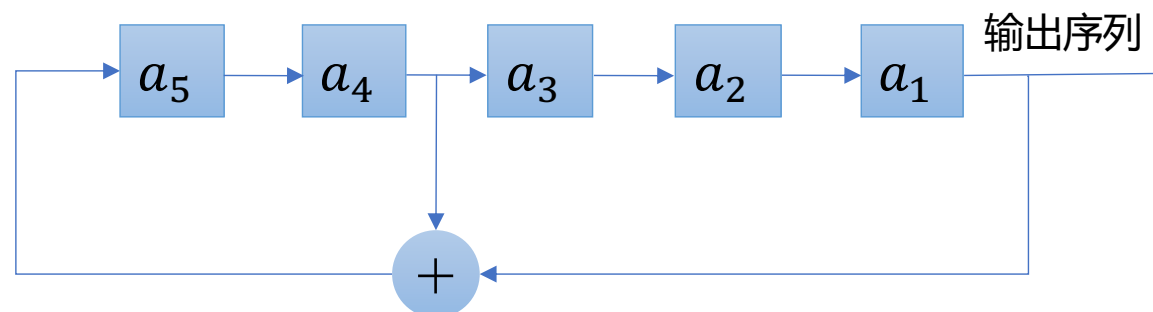


$$\begin{aligned}
 a_{n+1} &= c_n a_1 \oplus c_{n-1} a_2 \oplus \dots \oplus c_1 a_n \\
 a_{n+2} &= c_n a_2 \oplus c_{n-1} a_3 \oplus \dots \oplus c_1 a_{n+1}
 \end{aligned}$$

- **例** 如图是一个5级线性反馈移位寄存器

初始状态为:  $(a_1, a_2, a_3, a_4, a_5) = (1, 0, 0, 1, 1)$

反馈函数:  $f(x_1, x_2, x_3, x_4, x_5) = x_1 \oplus x_4$



| 时刻 | 0             | 1             | 2             | 3             | 4             | 5             |
|----|---------------|---------------|---------------|---------------|---------------|---------------|
| 状态 | 1, 0, 0, 1, 1 | 0, 0, 1, 1, 0 | 0, 1, 1, 0, 1 | 1, 1, 0, 1, 0 | 1, 0, 1, 0, 0 | 0, 1, 0, 0, 1 |
| 输出 | 1             | 0             | 0             | 1             | 1             | 0             |

可求出输出序列为 1001101001000010101110110001111100110...,

周期为  $31 = 2^5 - 1$ 。

### □ 线性反馈移位寄存器LFSR (linear feedback shift register)

- n级线性反馈移位寄存器最多有 $2^n$ 个不同的状态。
  - 初始状态为零，则其状态恒为零。
  - 若其初始状态非零，则其后继状态不会为零。
- 因此 n 级LFSR的状态周期 $\leq 2^n - 1$
- LFSR输出序列的性质：完全由其反馈函数决定
- 输出序列的周期=状态周期， $\leq 2^n - 1$
- 选择合适的反馈函数可使序列的周期达到最大值  $2^n - 1$ ，周期达到最大值的序列称为 m 序列。

### □ m序列的伪随机性\*

- 流密码的安全性取决于密钥流的安全性，要求密钥流序列有好的随机性，以使密码分析者对它无法预测。也就是说，即使截获其中一段，也无法推测后面是什么。
- 如果密钥流是周期的，要完全做到随机性是困难的。严格地说，这样的序列不可能做到随机，只能要求截获比周期短的一段时不会泄露更多信息，这样的序列称为伪随机序列。
- m序列是一种典型的伪随机序列。在通信领域有着广泛的应用，如扩频通信、卫星通信的码分多址（CDMA），数字数据中的加密、加扰、同步、误码率测量等领域。

### □ m序列的伪随机性\*

- **定义：** GF(2)上周期为T的序列 $\{a_i\}$ 的自相关函数定义为

$$R(\tau) = \frac{1}{T} \sum_{k=1}^T (-1)^{a_k} (-1)^{a_{k+\tau}}, 0 \leq \tau \leq T - 1$$

当 $\tau = 0$ 时,  $R(\tau) = 1$ ;

当 $\tau \neq 0$ 时, 称 $R(\tau)$ 为异相自相关函数。

### □ m序列的伪随机性\*

- Golomb对伪随机周期序列提出了应满足的如下 3 个随机性公设：

- 在序列的一个周期内，0 与 1 的个数相差至多为 1；

说明 $\{a_i\}$ 中0与1出现的概率基本上相同。

- 在序列的一个周期内，长为 $i$ 的游程占游程总数的  $1/2^i$  ( $i=1,2,\dots$ )，且在等长的游程中 0 的游程个数和 1 的游程个数相等。

说明 0 与 1 在序列中每一位置上出现的概率相同。

- 异相自相关函数是一个常数。

意味着通过对序列与其平移后的序列做比较，不能给出其他任何信息。

## □ m序列的伪随机性\*

- m 序列满足 Golomb 的 3 个随机性公设。
- **定理** GF(2)上的n长m序列 $\{a_i\}$ 具有如下性质:
  - 在一个周期内, 0、1出现的次数分别为 $2^{n-1} - 1$ 和 $2^{n-1}$ 。
  - 在一个周期内, 总游程数为 $2^{n-1}$ ; 对 $1 \leq i \leq n-2$ , 长为i的游程有 $2^{n-i-1}$ 个, 且0、1游程各半; 长为n-1的0游程一个, 长为n的1游程一个。
  - $\{a_i\}$ 的自相关函数为

$$R(\tau) = \begin{cases} 1, & \tau = 0 \\ -\frac{1}{2^n - 1}, & 0 < \tau \leq 2^n - 2 \end{cases}$$

## □ m序列密码的破译

- 设滚动密钥流生成器是线性反馈移位寄存器，产生的密钥是m序列。

$$S_h = \begin{pmatrix} a_h \\ a_{h+1} \\ \dots \\ a_{h+n-1} \end{pmatrix}, S_{h+1} = \begin{pmatrix} a_{h+1} \\ a_{h+2} \\ \dots \\ a_{h+n} \end{pmatrix}$$

- 设序列 $\{a_i\}$ 满足线性递推关系：

$$a_{h+n} = c_1 a_{h+n-1} \oplus c_2 a_{h+n-2} \oplus \dots \oplus c_n a_h$$

可表示为 
$$S_{h+1} = \begin{pmatrix} a_{h+1} \\ a_{h+2} \\ \dots \\ a_{h+n} \end{pmatrix} = \begin{pmatrix} 0 & 1 & 0 & \dots & 0 \\ 0 & 0 & 1 & \dots & 0 \\ \dots & \dots & \dots & \dots & \dots \\ c_n & c_{n-1} & c_{n-2} & \dots & c_1 \end{pmatrix} \begin{pmatrix} a_h \\ a_{h+1} \\ \dots \\ a_{h+n-1} \end{pmatrix} = S_h$$



## □ m序列密码的破译

- 又设敌手知道一段长为 $2n$ 的明文-密文对, 即已知:

$$x = x_1 x_2 \cdots x_{2n}$$

$$y = y_1 y_2 \cdots y_{2n}$$

- 设 $z_i = x_i \oplus y_i$ , 可得一段长为 $2n$ 的密钥序列  $z = z_1 z_2 \cdots z_{2n}$

- 则LFSR输出序列的连续 $n+1$ 个状态:

$$S_1 = (z_1 z_2 \cdots z_n) \overset{\text{记为}}{=} (a_1 a_2 \cdots a_n)$$

$$S_2 = (z_2 z_3 \cdots z_{n+1}) \overset{\text{记为}}{=} (a_2 a_3 \cdots a_{n+1})$$

...

$$S_{n+1} = (z_{n+1} z_{n+2} \cdots z_{2n}) \overset{\text{记为}}{=} (a_{n+1} a_{n+2} \cdots a_{2n})$$

## □ m序列密码的破译

- 设矩阵  $X = (S_1 \ S_2 \ \cdots \ S_n)$ ,

$$\begin{aligned}(a_{n+1} \ a_{n+2} \ \cdots \ a_{2n}) &= (c_n \ c_{n-1} \ \cdots \ c_1) \begin{pmatrix} a_1 & a_2 & \cdots & a_n \\ a_1 & a_2 & \cdots & a_{n+1} \\ & & \cdots & \\ a_n & a_{n+1} & \cdots & a_{2n-1} \end{pmatrix} \\ &= (c_n \ c_{n-1} \ \cdots \ c_1) X\end{aligned}$$

- 则  $(c_n \ c_{n-1} \ \cdots \ c_1) = (a_{n+1} \ a_{n+2} \ \cdots \ a_{2n}) X^{-1}$

- **例** 设敌手得到密文串: 101101011110010

和相应的明文串: 011001111111001

相应的密钥流: 110100100001011

进一步假定敌手还知道密钥流是使用 5 级线性反馈移位寄存器产生的, 那么敌手可分别用密文串中的前 10 个比特和明文串中的前 10 个比特建立如下方程

$$(a_6 \ a_7 \ a_8 \ a_9 \ a_{10}) = (c_5 \ c_4 \ c_3 \ c_2 \ c_1) \begin{pmatrix} a_1 & a_2 & a_3 & a_4 & a_5 \\ a_2 & a_3 & a_4 & a_5 & a_6 \\ a_3 & a_4 & a_5 & a_6 & a_7 \\ a_4 & a_5 & a_6 & a_7 & a_8 \\ a_5 & a_6 & a_7 & a_8 & a_9 \end{pmatrix}$$

即

$$(0\ 1\ 0\ 0\ 0) = (c_5\ c_4\ c_3\ c_2\ c_1) \begin{pmatrix} 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{pmatrix}$$

而

$$\begin{pmatrix} 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{pmatrix}^{-1} = \begin{pmatrix} 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 & 0 \end{pmatrix}$$



则

$$(c_5 \ c_4 \ c_3 \ c_2 \ c_1) = (0 \ 1 \ 0 \ 0 \ 0) \begin{pmatrix} 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 & 0 \end{pmatrix} = (1 \ 0 \ 0 \ 1 \ 0)$$

密钥流的递推关系为

$$a_{i+5} = c_5 a_i \oplus c_2 a_{i+3} = a_i \oplus a_{i+3}$$

### □ 美国制定数据加密标准简况

通信与计算机相结合是人类步入信息社会的一个阶梯，它始于六十年代末，完成于90年代初。计算机通信网的形成与发展，要求信息作业标准化，安全保密亦不例外。只有标准化，才能真正实现网的安全，才能推广使用加密手段，以便于训练、生产和降低成本。美国 NBS (National Bureau of Standards) 在 1973 年 5 月 15 日公布了征求建议。1974 年 8 月 27 日 NBS 再次出公告征求建议，对建议方案提出如下要求：

- (1) 算法必须完全确定而无含糊之处；
- (2) 算法必须有足够高的保护水准，即可以检测到威胁，恢复密钥所必须的运算时间或运算次数足够大；
- (3) 保护方法必须只依赖于密钥的保密；
- (4) 对任何用户或产品供应者必须是不加区分的。

- IBM 公司从 60 年代末即看到通信网对于这种加密标准算法的需求，投入了相当的研究力量开发，成立了以 Tuchman 博士为领导的小组，包括 A. Konkeim , E. Grossman, N. Coppersmith 和 L. Smith 等 (后二人做实现工作) 的研究新密码体制的小组，H. Fistel 进行设计，并在 1971 年完成的 LUCIFER 密码 (64 bit 分组，代换-置换，128 bit 密钥) 的基础上，改进成为建议的 DES 体制。NSA (National Standards Association) 组织有关专家对 IBM 的算法进行了鉴定，而成为 DES 的基础。
- 1975 年 3 月 17 日 NBS 公布了这个算法，并说明要以它作为联邦信息处理标准，征求各方意见。1977 年 1 月 15 日建议被批准为联邦标准 [FIPS PUB 46]，并设计推出 DES 芯片。DES 开始在银行、金融界广泛应用。1981 年美国 ANSI 将其作为标准，称之为 DEA [ANSI X3.92]。1983 年国际标准化组织 (ISO) 采用它作为标准，称作 DEA-1。

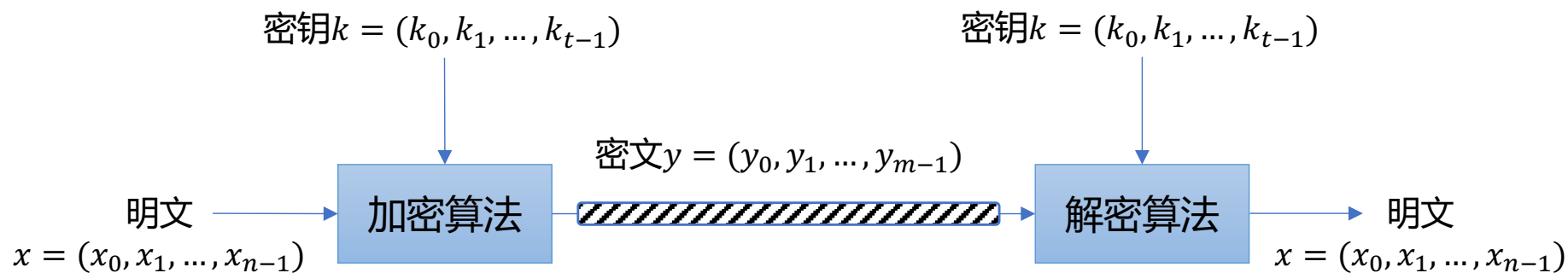
- 1984 年 9 月美国总统签署145号国家安全决策令 (NSDD), 命令 NSA 着手发展新的加密标准, 用于政府系统非机密数据和私人企事业单位。NSA宣布每隔5 年重新审议 DES 是否继续作为联邦标准, 1988年 (FIPS46-1)、1993 年 (FIPS46-2), 1998年不再重新批准 DES 为联邦标准。虽然 DES 不会长期地作为数据加密标准算法, 但它仍是**迄今为止得到最广泛应用的一种算法, 也是一种最有代表性的分组加密体制**。因此, 详细地研究这一算法的基本原理、设计思想、安全性分析以及实际应用中的有关问题, 对于掌握分组密码理论和当前的实际应用都很有意义。
- 1993 年 4 月, Clinton 政府公布了一项建议的加密技术标准, 称作密钥托管加密技术标准 EES (Escrowed Encryption Standard)。其开发设计始于 1985年, 由 NSA 负责研究, 1990 年完成评价工作。其算法为SKIPJACK, 已由 Mykotronix 公司开发芯片产品, 编程后为 MYK-78 (26美元/片)。算法属美国政府 SECRET 密级。但安全性与算法是否公开无关。



- DES 发展史确定了发展公用标准算法模式，而 EES 的制定路线与 DES 的背道而驰。虽然，由美国政府精心选定的五人小组评估意见，不能解除人们对算法安全性的疑虑。人们怀疑有陷门和政府部门肆意侵犯公民权利。此举遭到广为反对，1995 年 5 月 AT&T Bell Lab 的 M. Blaze 博士在 PC 机上用 45 分钟时间使 SKIPJACK 的 LEAF 协议失败，伪造 ID 码获得成功。虽 NSA 声称已弥补，但丧失了公众对此体制的信心。1995 年 7 月美国政府宣布放弃用 EES 来加密数据，只将它用于语音通信。
- 重新回到制定 DES 标准立场。1997 年 1 月美国 NIST 着手进行 AES (Advanced Encryption Standard) 的研究，成立了标准工作室。1997 年 4 月 15 日讨论了 AES 的评估标准，开始在世界范围征集 AES 的建议算法，截止时间为 1998 年 6 月 15 日。1998 年 8 月 20~22 日经评审选定并公布了 15 个候选算法。

## □ 分组密码概述

- **分组密码**是将明文消息编码表示后的数字序列  $x_1, x_2, \dots, x_i, \dots$  划分成长为  $n$  的组  $x = (x_0, x_1, \dots, x_{n-1})$ , 各组分别在密钥  $k = (k_0, k_1, \dots, k_{t-1})$  控制下变换成等长的输出数字序列  $y = (y_0, y_1, \dots, y_{m-1})$  (长为  $m$  的矢量)。一般情况下,  $m = n$ 。
- 分组密码与流密码的不同之处在于输出的每一位数字不是只与相应时刻输入的明文数字有关, 而是与一组长度为  $n$  的明文数字有关。

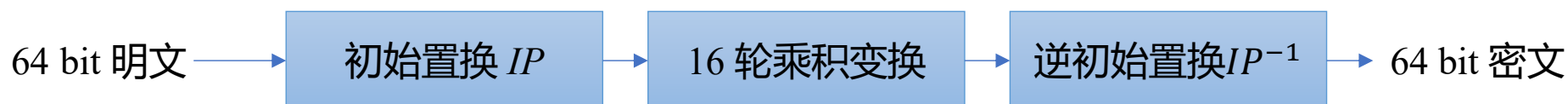


## □ 分组密码概述

- 分组密码的设计问题在于找到一种算法，能在密钥控制下从一个足够大且足够好的置换子集中，简单而迅速地选出一个置换，用来对当前输入的明文的数字组进行加密变换。设计的算法应满足下述要求：
  - 分组长度要足够大，防止明文穷举攻击奏效。
  - 密钥量要足够大，尽可能消除弱密钥并使所有密钥同等地好，以防止密钥穷举攻击奏效。
  - 由密钥确定置换的算法要足够复杂，充分实现明文与密钥的扩散和混淆，没有简单的关系可循，要能抗击各种已知的攻击。
  - 加密和解密运算简单，易于软件和硬件高速实现。
  - 数据扩展尽可能小。一般无数据扩展，在采用同态置换和随机化加密技术时可引入数据扩展。
  - 差错传播尽可能地小。

## □ DES 概述

- DES (Data Encryption Standard) 是迄今为止世界上最为广泛使用和流行的一种分组密码算法，它的分组长度为 64 比特，密钥长度为 56 比特。
- DES 加密算法的流程如下，有 3 个阶段：
  1. 初始置换  $IP$ ，用于重排明文分组的 64 bit 数据。
  2. 具有相同功能的 16 轮乘积变换，每轮中都有置换和代换运算，第 16 轮变换的输出分为左右两半，并被交换次序。
  3. 逆初始置换  $IP^{-1}$ （为  $IP$  的逆）从而产生 64 bit 的密文。



### □ 初始置换和逆初始置换

- **初始置换 $IP$** ：将 64 bit 明文的位置进行置换，得到一个乱序的 64 bit 明文组，而后分成左右两段，每段为 32 bit，以  $L_0$  和  $R_0$  表示， $IP$  中各列元素位置号数相差为 8，相当于将原明文各字节按列写出，各列比特经过偶采样和奇采样置换后，再对各行进行逆序。将阵中元素按行读出构成置换输出。
- **逆初始置换 $IP^{-1}$** ：将 16 轮迭代后给出的 64 bit 组进行置换，得到输出的密文组。输出为阵中元素按行读得的结果。
- $IP$  和  $IP^{-1}$  在密码意义上作用不大，因为输入组  $x$  与其输出组  $y = IP(x)$  (或  $IP^{-1}(x)$ ) 是已知的一一对应关系。它们的作用在于打乱原来输入  $x$  的 ASCII 码字划分关系，并将原来明文的校验位  $x_8, x_{16}, \dots, x_{64}$  变成成为  $IP$  输出的一个字节。

## Feistel网络

很多分组密码的结构从本质上说都是基于一个称为 Feistel网络的结构。

$k$ 是最初由Horst Feistel 在其设计的Lucifer算法中首次使用  
后来的许多算法都采用了这种结构或其变形

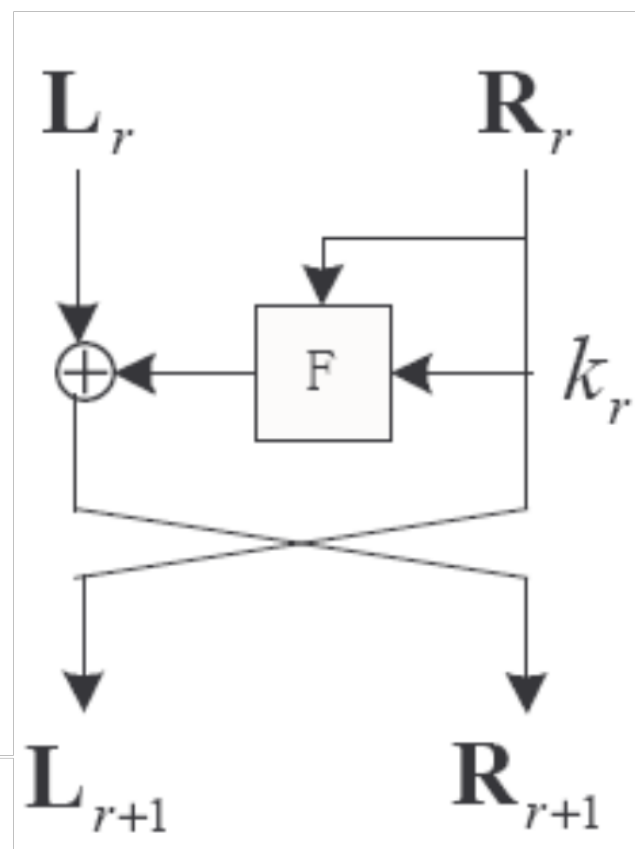


## 传统的Feistel网络

左右分组的长度相等,

每一轮中的 $F$ 函数是一致的,

每一轮中所有的输入都参与运算

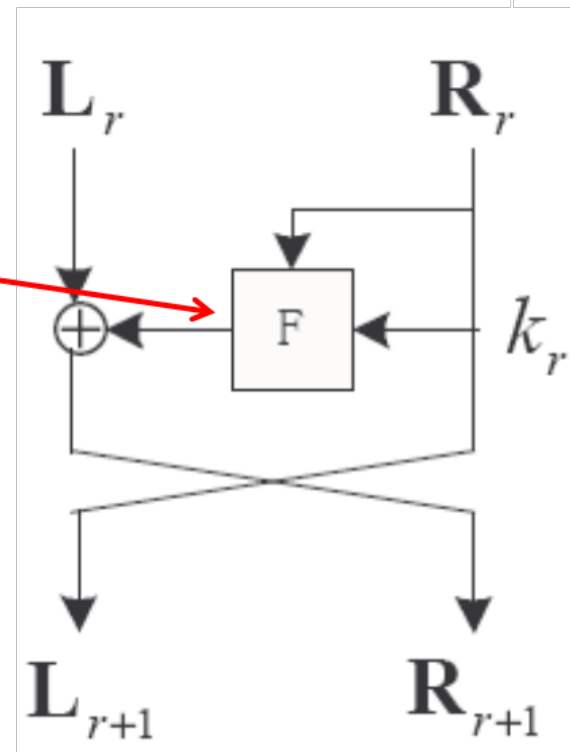


## Feistel网络的F-函数

Feistel网络的基本模块是F—函数：

一个依赖于密钥的把输入串映射到输出串的映射。 $F$ —函数通常是非线性且不可逆的。

$$F: \bar{K} \times \{0, 1\}^{n/2} \rightarrow \{0, 1\}^{n/2}$$





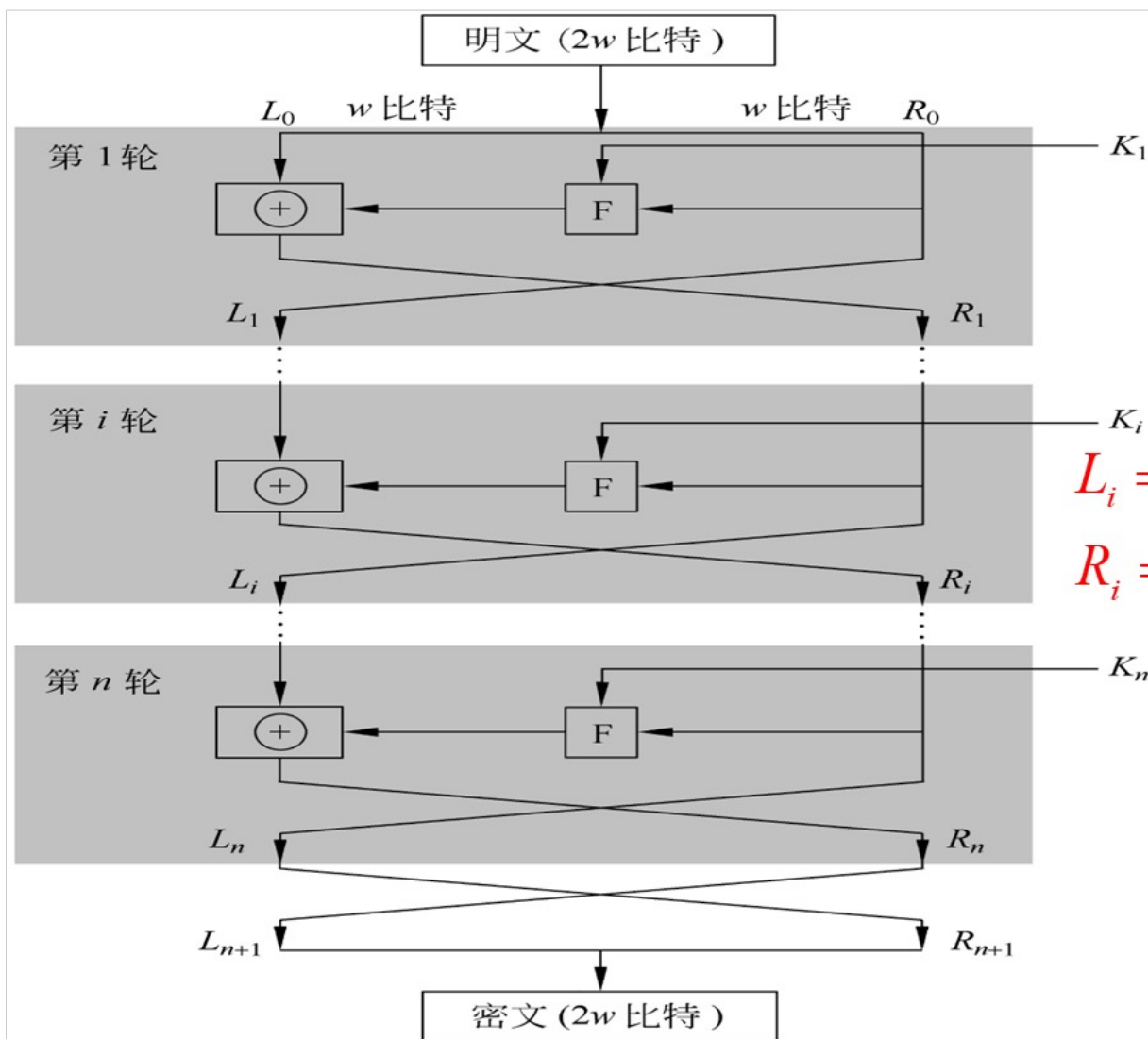
## Feistel密码结构

**乘积密码**指顺序地执行两个或多个基本密码系统,使得最后结果的密码强度高于每个基本密码系统产生的结果.

Feistel还提出了实现代换和置换的方法。其思想实际上是Shannon提出的**利用乘积密码实现混淆和扩散思想**的具体应用。



## Feistel网络示意图



$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

**Feistel网络的实现与以下参数和特性有关：**

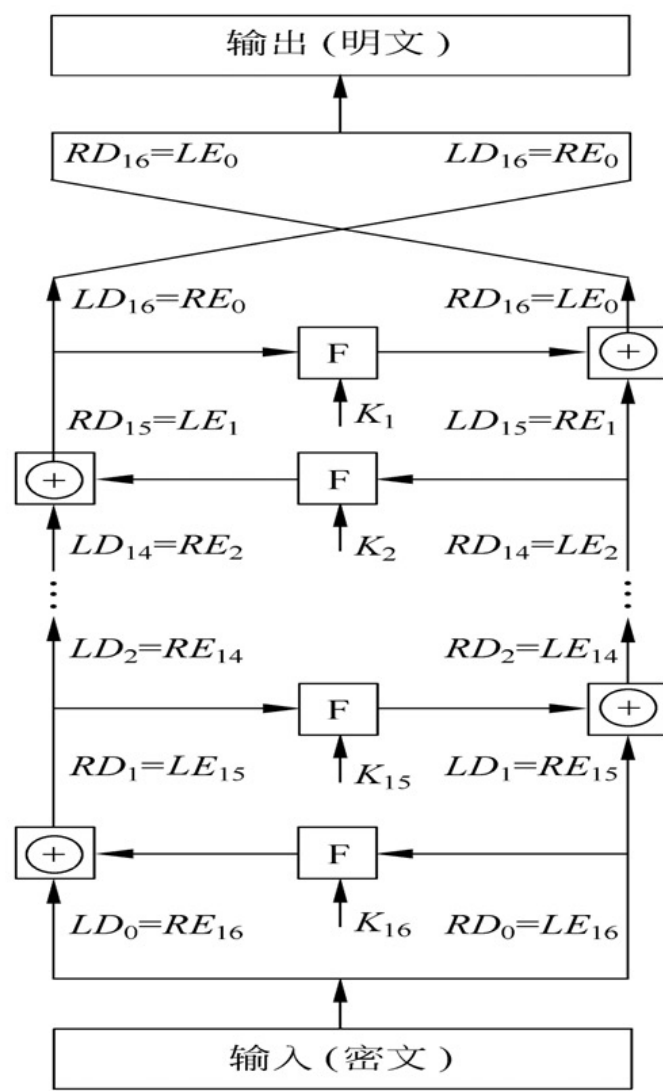
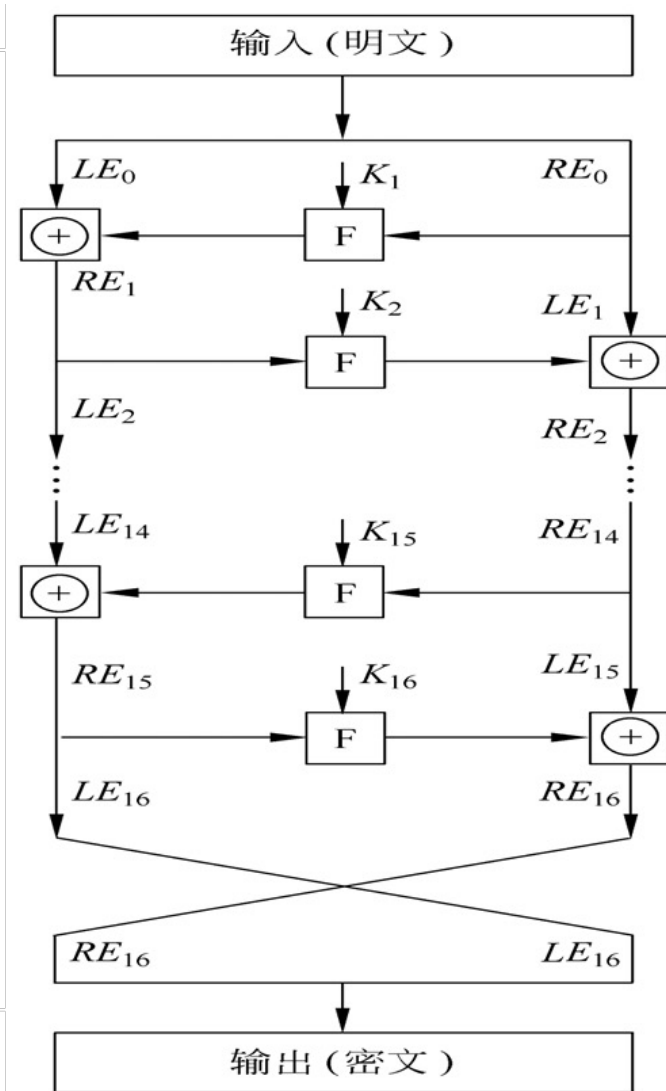
- ① **分组大小**：分组越大则安全性越高，但加密速度就越慢。
- ② **密钥大小**：密钥越长则安全性越高，但加密速度就越慢。
- ③ **轮数**：单轮结构远不足以保证安全性，但多轮结构可提供足够的安全性。DES轮数取为16。
- ④ **子密钥产生算法**：该算法的复杂性越大，则密码分析的困难性就越大。
- ⑤ **F函数**：F函数的复杂性越大，密码分析的困难性也越大。

## Feistel解密结构

Feistel解密过程本质上和加密过程是一样的,

算法使用密文作为输入, 但使用子密钥 $K_i$ 的次序与加密过程相反, 即第1轮使用 $K_n$ , 第2轮使用 $K_{n-1}$ , ... .., 最后一轮使用 $K_1$ 。

这一特性保证了解密和加密可采用同一算法。



Feistel加解密过程

Feistel网络能否直接用作分组密码算法？不能。

首先，明文 $(L(0), R(0))$ 和密文 $(L(1), R(1))$ 之间有一个明显的关系 $R(0) = L(1)$ ，即密钥只覆盖了明文的一半。

其次，设攻击者Eve获得了明文 $(L(0), R(0))$ 和密文 $(L(1), R(1))$ 。Eve知道 $R(1) = L(0) \oplus F(R(0), k)$ ，即 $F(R(0), k) = R(1) \oplus L(0)$ 。

在方程中，Eve仅仅不知道密钥 $k$ 。如果函数 $F$ 设计得不好，就可能在方程中解出密钥 $k$ ，或猜测出密钥 $k$ 。这就是说，算法的安全性完全依赖于函数 $F$ ，安全性依靠过于单薄。

## 多轮迭代与轮函数

**迭代型**分组密码就是用简单的、安全性弱的加密算法进行多轮迭代以获得强的安全性。单轮加密算法称为轮函数，所用的密钥称为单轮子密钥。

比如，把Feistel网络作为轮函数，将迭代型分组密码的加密算法构造如下：

Feistel网络 $\rightarrow$  Feistel网络 $\rightarrow \dots$  Feistel网络 $\rightarrow$ 左右对换。

则（1）克服了Feistel网络的安全性缺陷；（2）分组密码是“加解密相似的”



IBM的工作并不是密码学，但我与费斯特尔进行了许多讨论，这极大地开阔了我对之前没预料到的发展可能的眼界。IBM对发展商业应用密码学投资的事实也表明了此类工作的必要性和价值。

霍斯特·费斯特尔（见图）是一名在德国出生的IBM员工，他被认为是DES的创造者（尽管其他人也参与了这项工作—不久之后会有更多）。他想把该系统称为Dataseal,但是IBM使用了Demonstration Cipher这个词，简称为Demon。最后，名字变成了Lucifer,维持了费斯特尔称之为恶魔的“邪恶气氛”以及“cifer”（密码）：DSD-1是这个密码在内部使用时的另一个名字。Lucifer在20世纪70年代初被劳埃德伦敦银行用于现金分配系统。





## □ 初始置换和逆初始置换

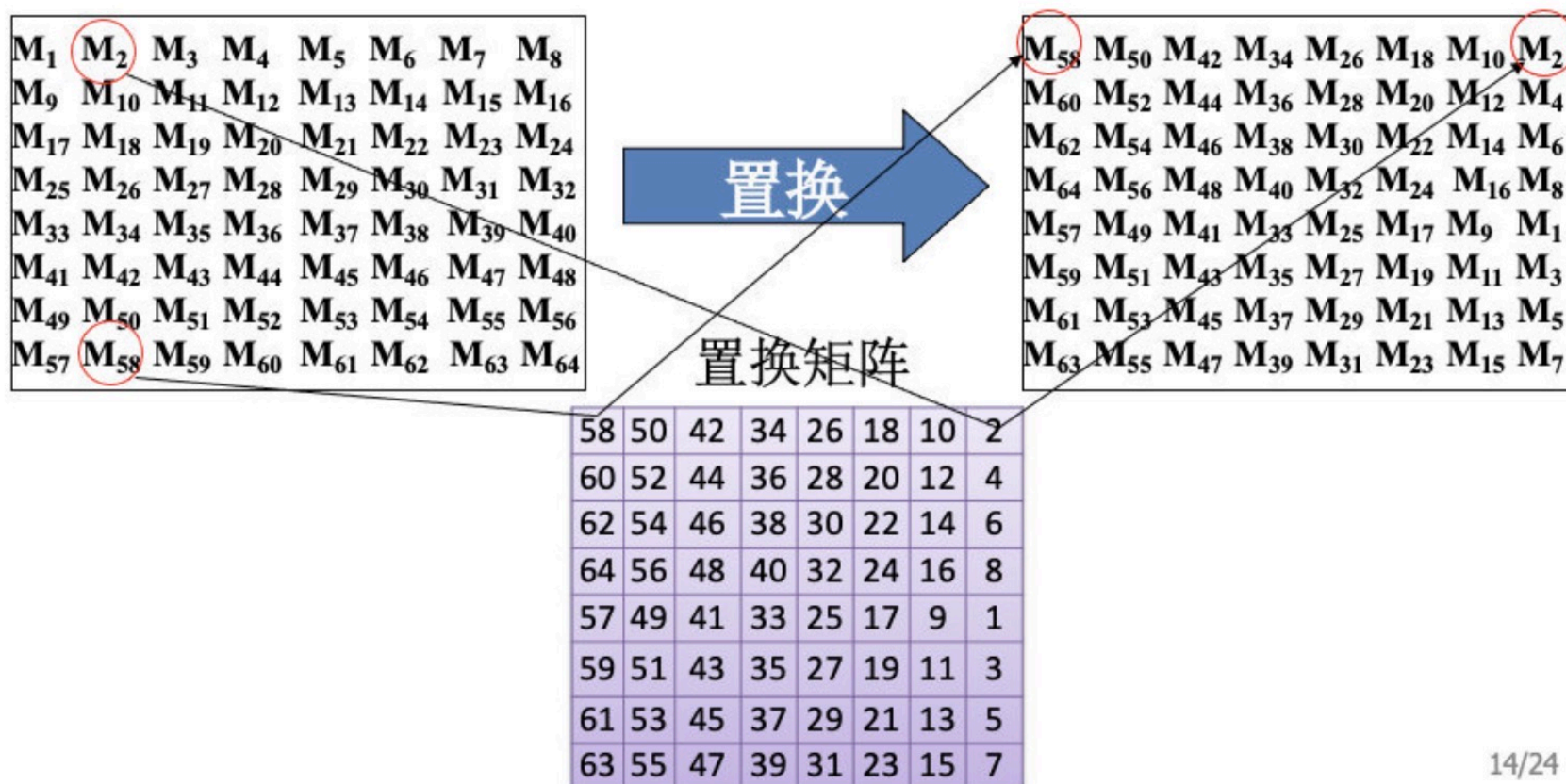
|    |    |    |    |    |    |    |   |
|----|----|----|----|----|----|----|---|
| 58 | 50 | 42 | 34 | 26 | 18 | 10 | 2 |
| 60 | 52 | 44 | 36 | 28 | 20 | 12 | 4 |
| 62 | 54 | 46 | 38 | 30 | 22 | 14 | 6 |
| 64 | 56 | 48 | 40 | 32 | 24 | 16 | 8 |
| 57 | 49 | 41 | 33 | 25 | 17 | 9  | 1 |
| 59 | 51 | 43 | 35 | 27 | 19 | 11 | 3 |
| 61 | 53 | 45 | 37 | 29 | 21 | 13 | 5 |
| 63 | 55 | 47 | 39 | 31 | 23 | 15 | 7 |

初始置换  $IP$

|    |   |    |    |    |    |    |    |
|----|---|----|----|----|----|----|----|
| 40 | 8 | 48 | 16 | 56 | 24 | 64 | 32 |
| 39 | 7 | 47 | 15 | 55 | 23 | 63 | 31 |
| 38 | 6 | 46 | 14 | 54 | 22 | 62 | 30 |
| 37 | 5 | 45 | 13 | 53 | 21 | 61 | 29 |
| 36 | 4 | 44 | 12 | 52 | 20 | 60 | 28 |
| 35 | 3 | 43 | 11 | 51 | 19 | 59 | 27 |
| 34 | 2 | 42 | 10 | 50 | 18 | 58 | 26 |
| 33 | 1 | 41 | 9  | 49 | 17 | 57 | 25 |

逆初始置换  $IP^{-1}$

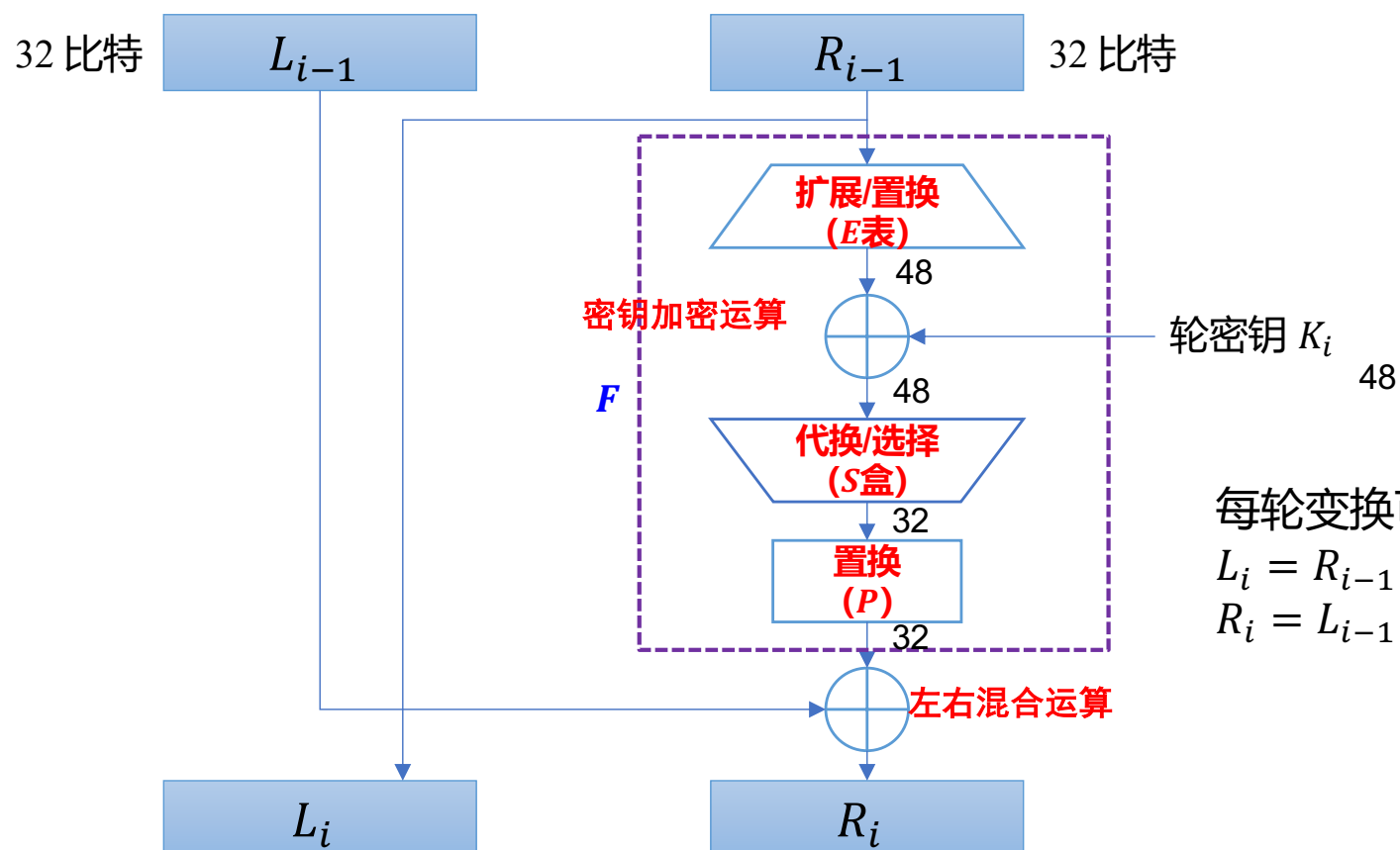
## □ 初始置换和逆初始置换



14/24

## □ 16 轮乘积变换 (Feistel 结构)

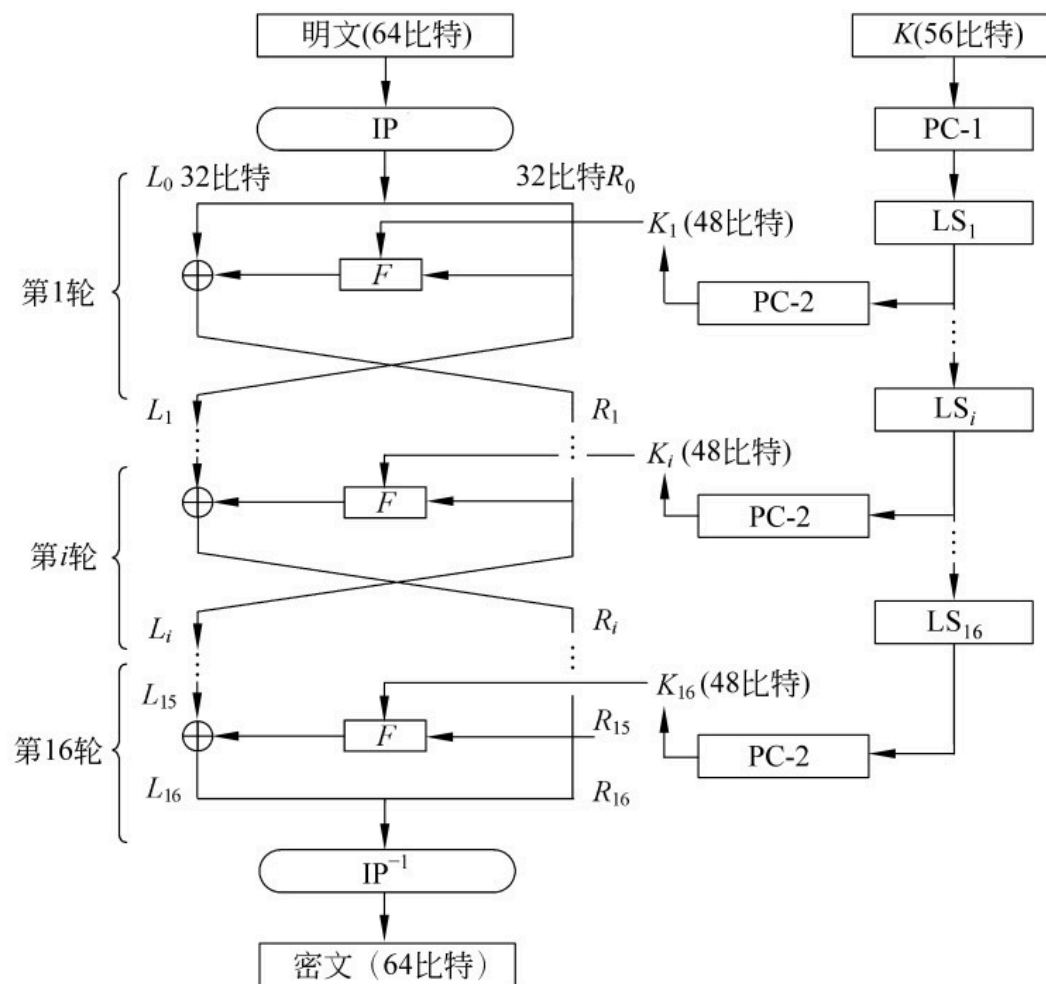
- 在每轮迭代中，64 位的中间结果被分成左右两部分，且作为相互独立的 32 位数据进行处理。每轮迭代的输入是上轮的结果  $L_{i-1}$  和  $R_{i-1}$ 。



每轮变换可由以下公式表示：

$$\begin{aligned}
 L_i &= R_{i-1} \\
 R_i &= L_{i-1} \oplus F(R_{i-1}, K_i)
 \end{aligned}$$

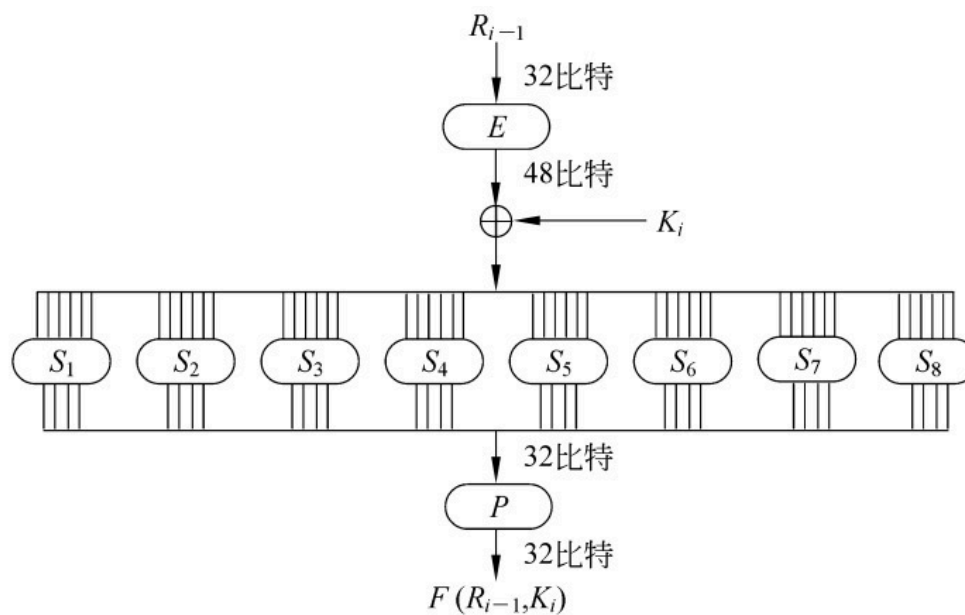
## □ 16 轮乘积变换 (Feistel 结构)



每轮变换可由以下公式表示：

$$\begin{aligned}
 L_i &= R_{i-1} \\
 R_i &= L_{i-1} \oplus F(R_{i-1}, K_i)
 \end{aligned}$$

### □ 16 轮乘积变换 (Feistel 结构)



## □ 16 轮乘积变换 (Feistel 结构)

函数  $F$

- **选择扩展运算  $E$ :** 将 32 bit 输入  $R_{i-1}$  扩展成 48 bit 输出。
- **密钥加密运算:** 将子密钥产生器输出的 48 bit 子密钥  $k_i$  与选择扩展运算  $E$  输出的 48 bit 数据按位模 2 相加。
- **选择压缩运算  $S$ :** 将前面送来的 48 bit 数据自左至右分成 8 组, 每组 6 bit。而后并行送入 8 个  $S$  盒, 每个  $S$  盒为一非线性代换网络, 有 4 bit 输出。
- **置换运算  $P$ :** 对  $S_1$  至  $S_8$  盒输出的 32 bit 数据进行坐标置换。
- **左右混合运算:** 置换  $P$  输出的 32 bit 数据与左边 32 bit 即  $L_{i-1}$  逐位模 2 相加, 所得到的 32 bit 作为下一轮迭代用的右边的数字段。并将  $R_{i-1}$  并行送到左边的寄存器, 作为下一轮迭代用的左边的数字段。

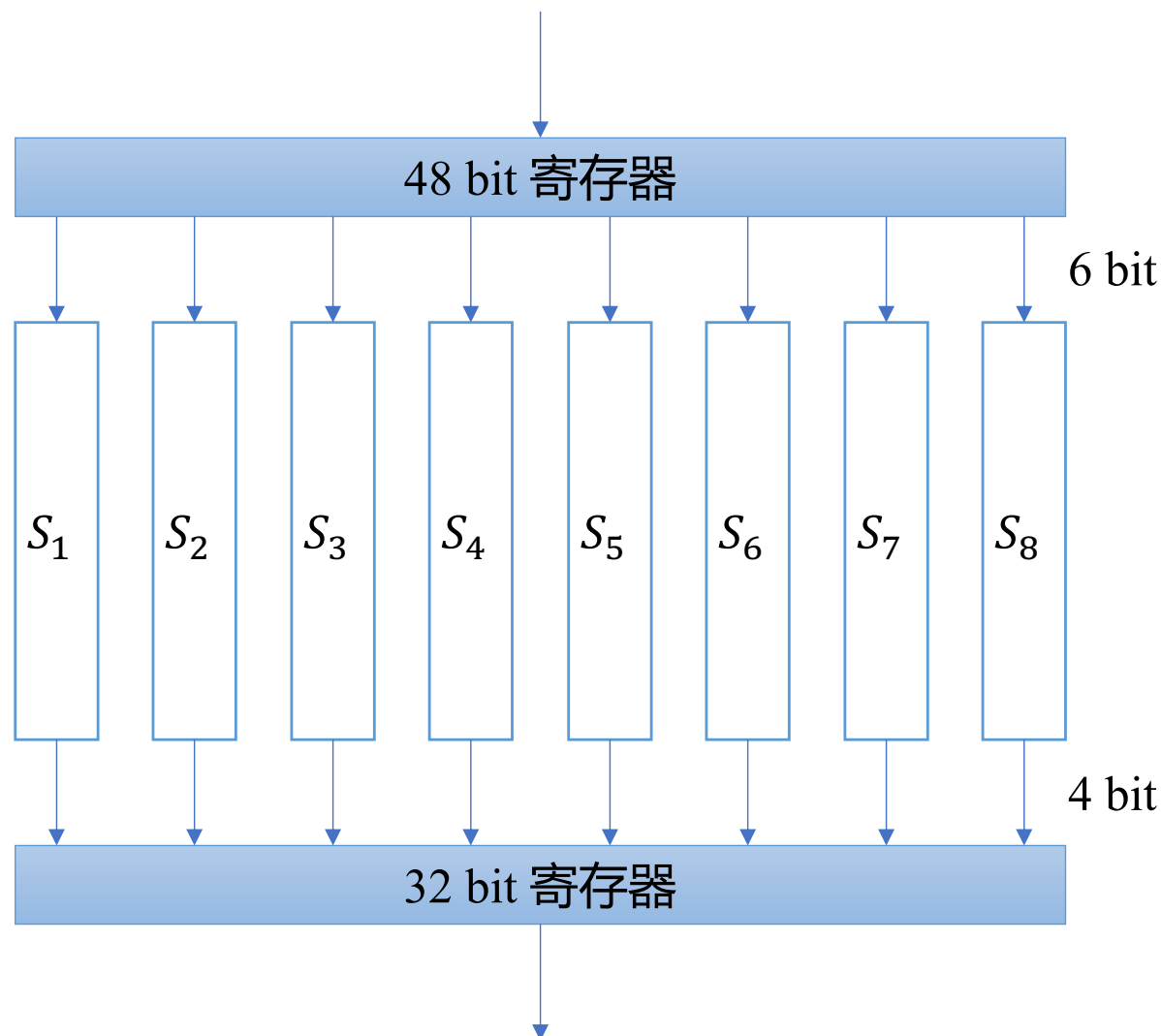
## □ 选择扩展运算 E

- 令  $s$  表示  $E$  原输入数据比特的原下标, 则  $E$  的输出是将原下标  $s \equiv 0$  或  $1(\text{mod}4)$  的各比特重复一次得到的, 即对原第 32, 1, 4, 5, 8, 9, 12, 13, 16, 17, 20, 21, 24, 25, 28, 29 各位都重复一次, 实现数据扩展。将表中数据按行读出得到 48 bit 输出。

|    |    |    |    |         |    |    |    |    |    |    |
|----|----|----|----|---------|----|----|----|----|----|----|
| 1  | 2  | 3  | 4  | 扩展<br>→ | 32 | 1  | 2  | 3  | 4  | 5  |
| 5  | 6  | 7  | 8  |         | 4  | 5  | 6  | 7  | 8  | 9  |
| 9  | 10 | 11 | 12 |         | 8  | 9  | 10 | 11 | 12 | 13 |
| 13 | 14 | 15 | 16 |         | 12 | 13 | 14 | 15 | 16 | 17 |
| 17 | 18 | 19 | 20 |         | 16 | 17 | 18 | 19 | 20 | 21 |
| 21 | 22 | 23 | 24 |         | 20 | 21 | 22 | 23 | 24 | 25 |
| 25 | 26 | 27 | 28 |         | 24 | 25 | 26 | 27 | 28 | 29 |
| 29 | 30 | 31 | 32 |         | 28 | 29 | 30 | 31 | 32 | 1  |

### □ 选择压缩运算 S

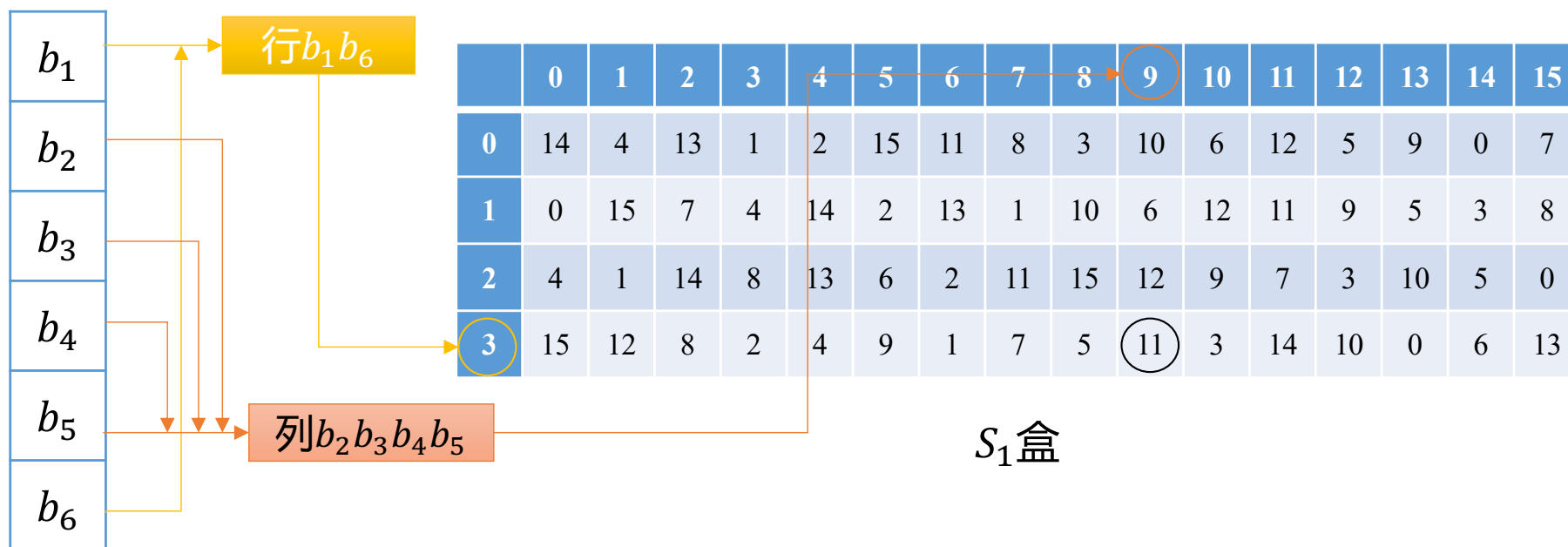
- 选择压缩运算由 8 个 S 盒构成，每个 S 盒都是 6 bit 输入，4 bit 输出。
- $S_i(b_1b_2b_3b_4b_5b_6)$  输出值是对应表  $S_i$  中  $(b_2b_3b_4b_5)$  列和  $(b_1b_6)$  行对应的值。
- S 盒对应于 Shannon 所提出的密码学思想中的混淆。





## □ 选择压缩运算 S

- 以将 110011 输入到  $S_1$  中为例：



$$\begin{array}{c}
 b_1 b_2 b_3 b_4 b_5 b_6 \\
 \hline
 110011
 \end{array}
 \begin{array}{c}
 \longrightarrow \\
 \text{行: } b_1 b_6 = (11)_2 = 3 \\
 \text{列: } b_2 b_3 b_4 b_5 = (1001)_2 = 9
 \end{array}
 \begin{array}{c}
 \longrightarrow \\
 \text{S}_1 \text{盒3行9列} \\
 \text{值: } 11 = (1011)_2
 \end{array}$$

## □ 置换运算 P

- 置换运算 P 对 8 个 S 盒的输出 (32 bit) 进行坐标置换, 对应于 Shannon 所提出的密码学思想中的扩散。

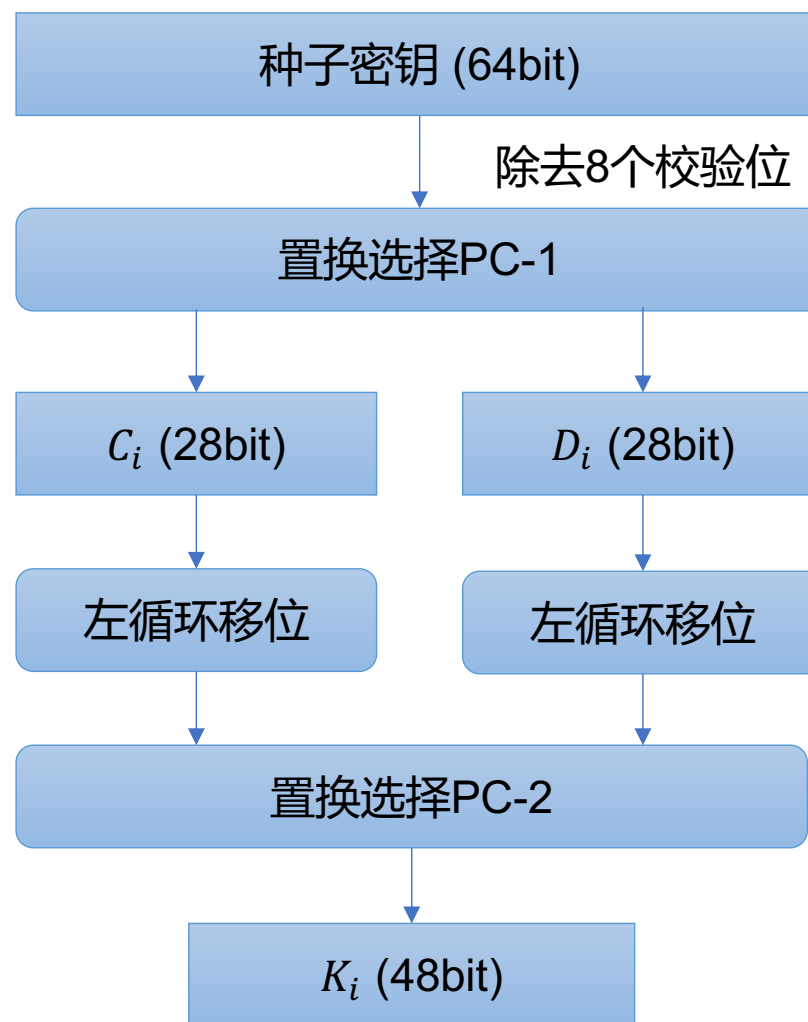
|    |    |    |    |         |    |    |    |    |
|----|----|----|----|---------|----|----|----|----|
| 1  | 2  | 3  | 4  | 置换<br>→ | 16 | 7  | 20 | 21 |
| 5  | 6  | 7  | 8  |         | 29 | 12 | 28 | 17 |
| 9  | 10 | 11 | 12 |         | 1  | 15 | 23 | 26 |
| 13 | 14 | 15 | 16 |         | 5  | 18 | 31 | 10 |
| 17 | 18 | 19 | 20 |         | 2  | 8  | 24 | 14 |
| 21 | 22 | 23 | 24 |         | 32 | 27 | 3  | 9  |
| 25 | 26 | 27 | 28 |         | 19 | 13 | 30 | 6  |
| 29 | 30 | 31 | 32 |         | 22 | 11 | 4  | 25 |

### □ 子密钥生成

**子密钥产生器：**将 64 bit 初始密钥经过置换选择 PC-1、循环移位置换、置换选择 PC-2 给出每次迭代加密用的子密钥  $K_i$ 。

- 在 64 bit 初始密钥中有 8 位为校验位，其位置号为 8、16、24、32、40、48、56 和 64。其余 56 位为有效位，用于子密钥计算。将这 56 位送入置换选择 PC-1。经过坐标置换后分成两组，每组为 28 bit，分别送入  $C$  寄存器和  $D$  寄存器中。
- 在各次迭代中， $C$  和  $D$  寄存器分别将存数进行左循环移位置换，所移位数与轮数有关。
- 每次移位后，将  $C$  和  $D$  寄存器原存数送给置换选择 PC-2。置换选择 PC-2 将  $C$  中第 9、18、22、25 位和  $D$  中第 7、10、15、26 位删去，并将其余数字置换位置后送出 48 bit 数字作为第  $i$  次迭代时所用的子密钥  $K_i$ 。

## □ 子密钥生成



## 子密钥生成

置换选择 PC-1

|    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|
| 57 | 49 | 41 | 33 | 25 | 17 | 9  |
| 1  | 58 | 50 | 42 | 34 | 26 | 18 |
| 10 | 2  | 59 | 51 | 43 | 35 | 27 |
| 19 | 11 | 3  | 60 | 52 | 44 | 36 |
| 63 | 55 | 47 | 39 | 31 | 23 | 15 |
| 7  | 62 | 54 | 46 | 38 | 30 | 22 |
| 14 | 6  | 61 | 53 | 45 | 37 | 29 |
| 21 | 13 | 5  | 28 | 20 | 12 | 4  |

置换选择 PC-2

|    |    |    |    |    |    |
|----|----|----|----|----|----|
| 14 | 17 | 11 | 24 | 1  | 5  |
| 3  | 28 | 15 | 6  | 21 | 10 |
| 23 | 19 | 12 | 4  | 26 | 8  |
| 16 | 7  | 27 | 20 | 13 | 2  |
| 41 | 52 | 31 | 37 | 47 | 55 |
| 30 | 40 | 51 | 45 | 33 | 48 |
| 44 | 49 | 39 | 56 | 34 | 53 |
| 46 | 42 | 50 | 36 | 29 | 52 |

左循环移位位数

| 轮数 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 |
|----|---|---|---|---|---|---|---|---|---|----|----|----|----|----|----|----|
| 位数 | 1 | 1 | 2 | 2 | 2 | 2 | 2 | 2 | 1 | 2  | 2  | 2  | 2  | 2  | 2  | 1  |

## □ 加解密过程

DES 算法加解密过程可总结如下：

- 加密: 式 (1) 和式 (2) 的运算进行 16 次后就得到密文组。

$$L_0 R_0 \leftarrow IP \left( \langle 64 \text{ bit 明文} \rangle \right)$$

$$L_i \leftarrow R_{i-1} \quad i = 1, \dots, 16 \quad (1)$$

$$R_i \leftarrow L_{i-1} \oplus F(R_{i-1}, k_i) \quad i = 1, \dots, 16 \quad (2)$$

$$\langle 64 \text{ bit 密文} \rangle \leftarrow IP^{-1}(R_{16}L_{16})$$

- 解密: DES 的加密运算是可逆的, 其解密过程可类似地进行。

$$R_{16} L_{16} \leftarrow IP \left( \langle 64 \text{ bit 密文} \rangle \right)$$

$$R_{i-1} \leftarrow L_i \quad i = 16, \dots, 1 \quad (3)$$

$$L_{i-1} \leftarrow R_i \oplus F(R_{i-1}, k_i) \quad i = 16, \dots, 1 \quad (4)$$

$$\langle 64 \text{ bit 明文} \rangle \leftarrow IP^{-1}(L_0 R_0)$$

### □ DES的安全性

- **密钥搜索攻击**：对 DES 安全性批评意见中，较为一致的看法是 DES 的密钥短。IBM 最初向 NBS 提交的建议方案采用 112 bit 密钥，但公布的 DES 标准采用 64 bit 密钥。有人认为 NSA 故意限制 DES 的密钥长度。
- DES 的密钥量为  $2^{56} = 72\,057\,594\,037\,927\,936 = 7.2 \times 10^{16}$  个。若要对 DES 进行密钥搜索破译，分析者在得到一组明文-密文对条件下，可对明文用不同的密钥加密，直到得到的密文与已知的明文-密文对中的相符，就可确定所用的密钥了。密钥搜索所需的时间取决于密钥空间的大小和执行一次加密所需的时间。若假定 DES 加密操作需时为  $100\mu\text{s}$  (一般微处理器能实现)，则搜索整个密钥空间需时为  $7.2 \times 10^{12}$  秒，近似为  $2.28 \times 10^5$  年。若以最快的 LSI 器件，DES 加密操作时间可降到  $5\mu\text{s}$ ，也要  $1.1 \times 10^4$  年才能穷尽密钥。但是由于差分和线性攻击法的出现以及计算技术的发展，按 Wiener 介绍，在 1993 年破译 DES 的费用为 100 万美元，需时 3 个半小时。如果将密钥加大到 80 bits，采用这类搜索机找出一个密钥所需的时间约为 6700 年。

### □ DES的历史地位

最终成为数据加密标准（Data Encryption Standard, DES）的密码已经被认为是不安全的<sup>1</sup>，但其依然很重要，源于以下几个原因：

- ① 它是几十年来的标准；
- ② 它利用了以前在非机密环境中看不到的技术，并为所有随后的密码设置了高标准；
- ③ 它源于 IBM 和美国国家安全局（NSA）之间为了一个公众可用的密码而进行的史无前例的秘密合作。美国国家安全局和工业界之间曾有过互动，但没有任何可以公开的内容。

DES 同样启发了马丁·E·赫尔曼，他将成为 20 世纪最重要的密码学家之一。他写道：“我将密码学的兴趣追溯到 3 个主要来源。”其中一个是大卫·卡恩的书《破译者》，另一个是香农 1949 年的论文<sup>2</sup>，他对第三部分的描述如下：

从 1968—1969 年，我在纽约约克敦高地的 IBM 沃森研究中心工作。我有一个同事叫霍斯特·费斯特尔，他因政府机密工作被引进，旨在推动 IBM 在密码学领域的研究。IBM 的工作于 1975 年在数据加密标准（DES）中达到了高潮。虽然我在 IBM 的工作并不是密码学，但我与费斯特尔进行了许



### □ DES的历史地位

#### DES扮演的突出角色

DES的结构闪烁着人类设计思想的精华，其优点和缺点被密码学界淋漓尽致地讨论，供后人借鉴。举例如下：

- (1) DES 使用了S盒，而S盒现在已经是几乎所有分组密码算法不可缺少的部件。
- (2) 迭代分组密码是分组密码的主流设计。
- (3) DES 轮函数结构是Feistel网络，这种结构现在已经是轮函数的经典结构之一。
- (4) DES算法的第一个和最后一个部件没有密钥的参与，在已知明文攻击之下不起任何安全性作用。以后设计的分组密码都纠正了这个缺点。
- (5) DES的S盒设计细节始终没有公布，因此被人们怀疑设有陷门（即密码设计者为自己预留的破译通道）。这种不透明的设计显然会影响其商用前景。这一缺点引出了商用分组密码设计的一个准则“透明性”，即密码的使用者能够确知该密码的安全强度。

### □ DES的历史地位

其次，对DES的各种攻击方法纷纷被提出。穷举搜索仍然是对分组密码实用攻击的主角。但进入90年代后，基于数学分析的攻击方法逐渐兴起，如以色列密码专家Shamir等人提出了差分密码分析，日本密码学家Matsui等人提出了线性密码分析，这些数学攻击方法都首先被用于破译DES，并且可以说取得了局部的成功。

(比如，有人在个人计算机上用差分密码分析方法，几分钟就破译了8轮DES（标准的DES算法为16轮迭代）。又比如，据称使用 $2^{43}$ 个明文/密文对就可以由线性密码分析方法破译DES。 $2^{43}$ 已经不是天文数字了，据称数十台工作站协同工作，经过十多天就可完成。)

- 为抵抗对DES的各种攻击，产生了强化版的三重DES。
- 三重DES的安全强度高于DES，但似乎不够简洁。
- 2000年5月，美国国家标准技术研究所（NIST）最终确定了DES的替代标准AES（Advanced Encryption Standard），才使DES基本上完成了自己的历史使命。AES的算法名称是Rijndael。

## □ 二重DES

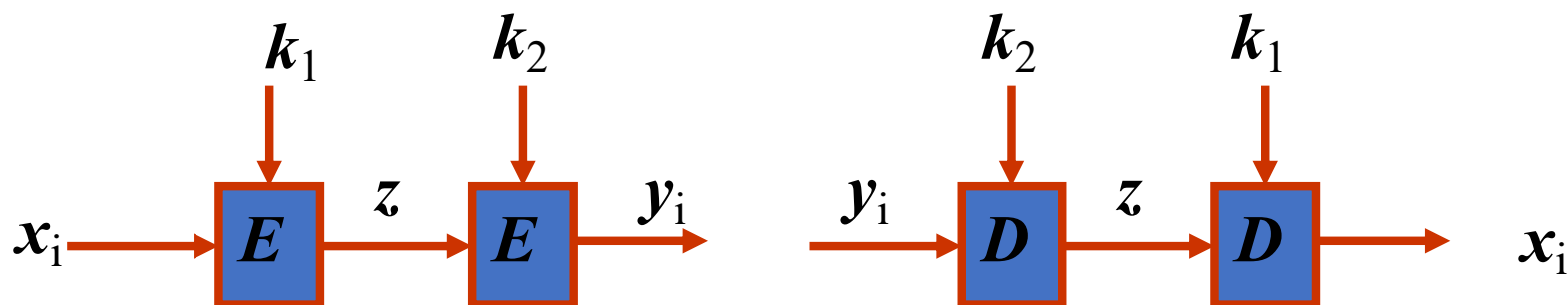
用DES进行两次加密，但这是否就意味着两重DES加密的强度等价于112 bit 密钥的密码的强度？答案是否定的。

### 中途相遇攻击法(Meet-in-the-Middle Attack)

由Diffie和Hellman[1977]最早提出，可以降低搜索量其基本想法如下。若有明文密文对 $(x_i, y_i)$ 满足

$$y_i = E_{k_2}[E_{k_1}[x_i]]$$

则可得  $z = E_{k_1}[x_i] = D_{k_2}[y_i]$



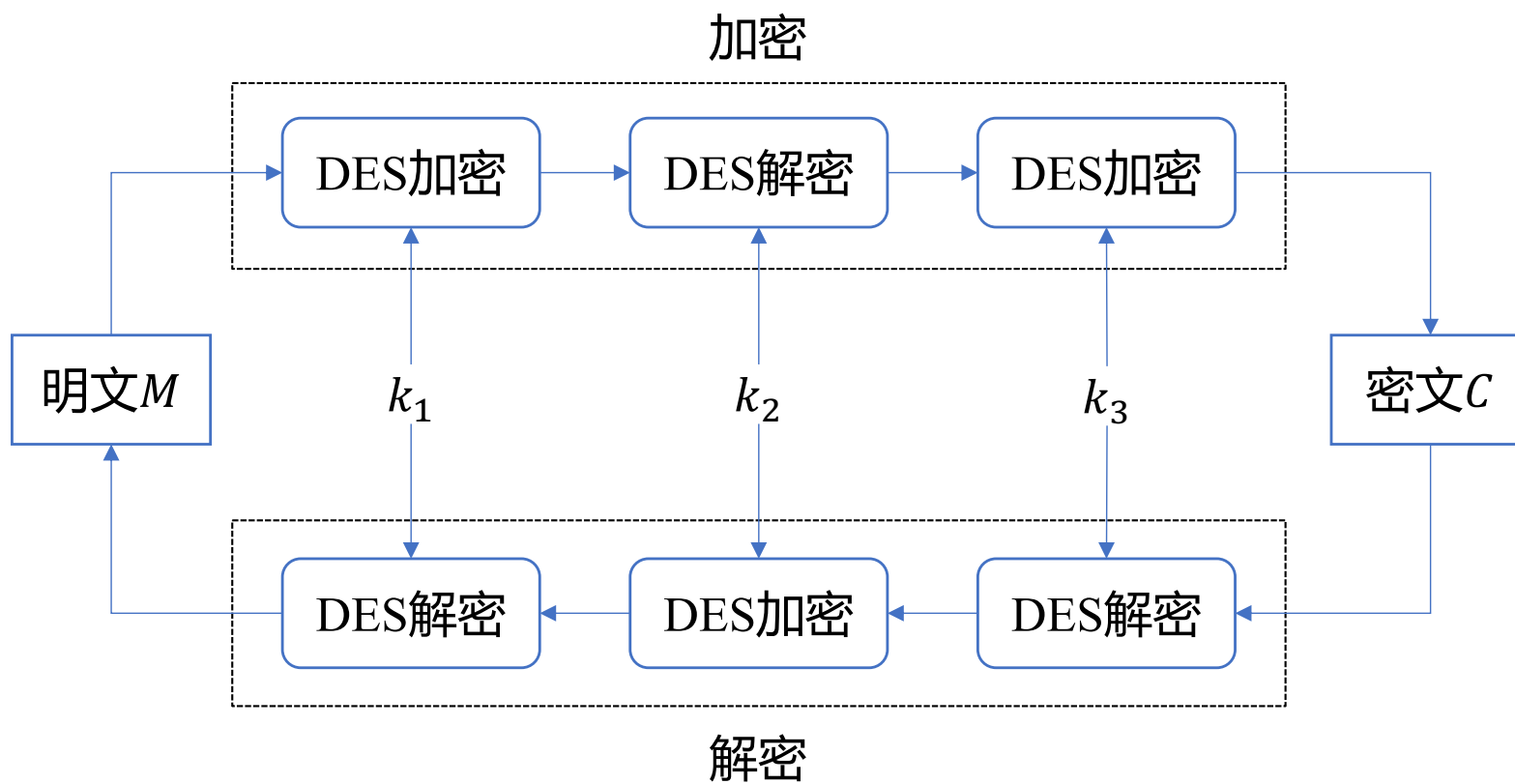
### □ 二重DES-中途相遇攻击

给定一已知明密文对 $(x_1, y_1)$ ，可按下述方法攻击。

- 以密钥 $k_1$ 的所有 $2^{56}$ 个可能的取值对此明文 $x_1$ 加密，并将密文 $z$ 存储在一个表中；
- 从所有可能的 $2^{56}$ 个密钥 $k_2$ 中依任意次序选出一个对给定的密文 $y_1$ 解密，并将每次解密结果 $z'$ 在上述表中查找相匹配的值。一旦找到，则可确定出两个密钥 $k_1$ 和 $k_2$ ；
- 以此对密钥 $k_1$ 和 $k_2$ 对另一已知明文密文对 $(x_2, y_2)$ 中的明文 $x_2$ 进行加密，如果能得出相应的密文 $y_2$ 就可确定 $k_1$ 和 $k_2$ 是所要找的密钥。

## □ 三重DES

为增强 DES 算法的安全性，人们提出许多DES的改进方案。**3DES** 是 DES的一个重要的改进算法，已在因特网的许多应用（如 **PGP** 和 **S/MIME**）中被采用。



3DES 加解密示意图

## □ AES概述

- 1997 年 4 月 5 日, NIST 发起征集 AES (Advanced Encryption Standard) 算法, 要求具有 128 比特的分组长度, 并支持 128、192 和 256 比特的密钥长度, 且要求 AES 能在全世界范围内免费使用。
- 1998 年 8 月 20 日, NIST 召开了第一次 AES 候选会议, 并公布了15个 AES候选算法。后来, NIST 又筛选出 5 个 AES 候选算法 (MARS, RC6, Rijindael, SERPENT, Twofish)。
- 2000 年 10 月, NIST 宣布由比利时的密码专家 Joan Daemen 博士和 Vincent Rijmen 博士开发的 [Rijndael 算法](#)做为 AES 的最终算法。
- AES 的设计原则:
  - 能抵抗所有已知的攻击;
  - 在各种平台上易于实现, 速度快;
  - 设计简单。

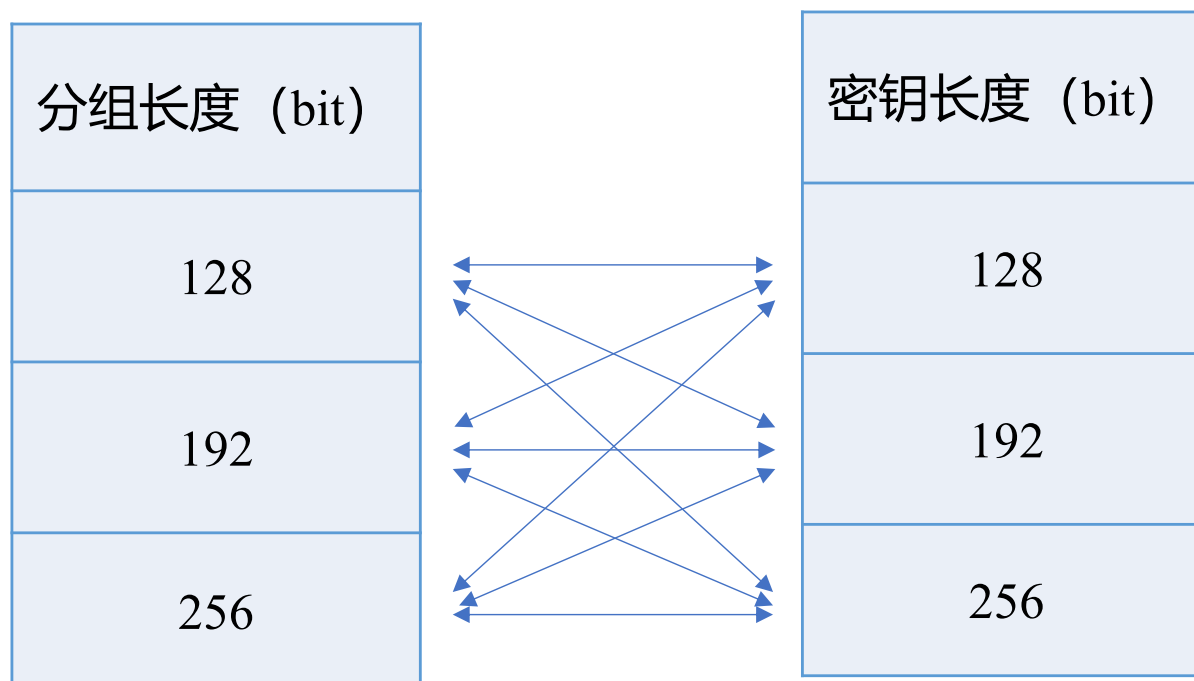
### □ AES数学基础

## RIJNDAEL的数学基础

- 有限域 $GF(2^8)$ ;
- 系数在 $GF(2^8)$ 上的多项式。

## □ AES概述

- AES 是一个迭代型分组密码，其分组长度和密钥长度都可变，各自可以独立的指定为 128 比特、192 比特、256 比特。



AES的分组长度和密钥长度的组合



## □ 状态、种子密钥和轮数

- **状态 (State)**: 密码运算中间结果称为状态, 状态用以字节为基本构成元素的矩阵阵列来表示, 该阵列有 4 行, 列数记为  $N_b$ :

$$N_b = \text{分组长度 (bits)} \div 32$$

$N_b$  取 4, 6, 8 对应分组长度 128, 192, 256 比特。算法的输入以字节为单位按  $a_{00}a_{10}a_{20}a_{30}a_{01}a_{11}a_{21}a_{31} \dots$  的顺序放置到状态阵列中。

- **种子密钥**: 种子密钥类似地用一个 4 行的矩阵阵列来表示, 列数记为  $N_k$ :

$$N_k = \text{密钥长度 (bits)} \div 32$$

$N_k$  可以取的值为 4, 6, 8; 对应的密钥长度为 128, 192, 256 比特。种子密钥以字节为单位按  $k_{00}k_{10}k_{20}k_{30}k_{01}k_{11}k_{21}k_{31} \dots$  的顺序放置到种子密钥阵列中。

- **轮数**: 算法迭代的轮数记为  $N_r$ ;  $N_r$  与  $N_b$  和  $N_k$  有关。

## □ 状态、种子密钥和轮数

- 以  $N_b = 6$  和  $N_k = 4$  为例，状态和种子密钥的矩阵阵列表示如下：

|          |          |          |          |          |          |
|----------|----------|----------|----------|----------|----------|
| $a_{00}$ | $a_{01}$ | $a_{02}$ | $a_{03}$ | $a_{04}$ | $a_{05}$ |
| $a_{10}$ | $a_{11}$ | $a_{12}$ | $a_{13}$ | $a_{14}$ | $a_{15}$ |
| $a_{20}$ | $a_{21}$ | $a_{22}$ | $a_{23}$ | $a_{24}$ | $a_{25}$ |
| $a_{30}$ | $a_{31}$ | $a_{32}$ | $a_{33}$ | $a_{34}$ | $a_{35}$ |

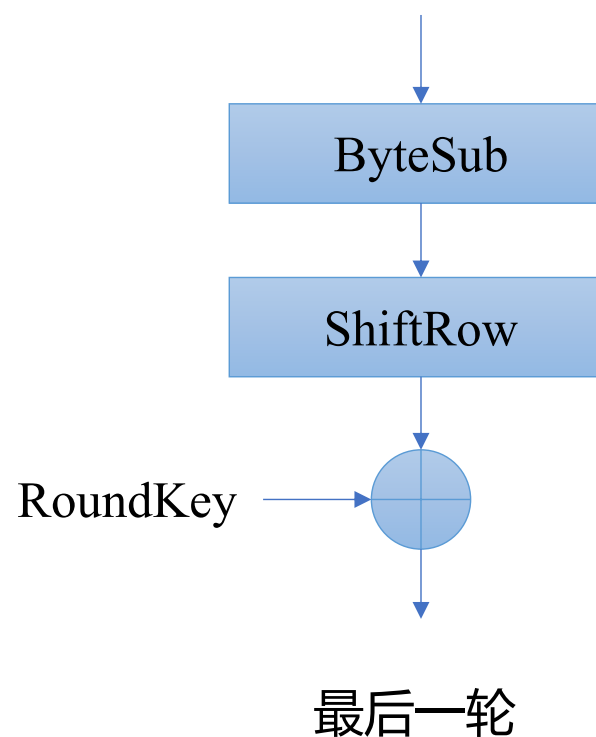
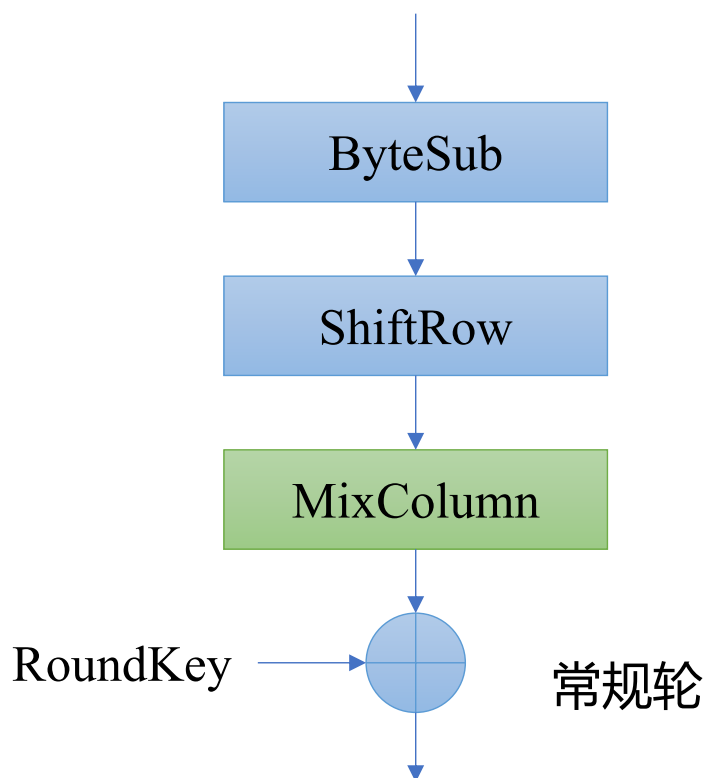
|          |          |          |          |
|----------|----------|----------|----------|
| $k_{00}$ | $k_{01}$ | $k_{02}$ | $k_{03}$ |
| $k_{10}$ | $k_{11}$ | $k_{12}$ | $k_{13}$ |
| $k_{20}$ | $k_{21}$ | $k_{22}$ | $k_{23}$ |
| $k_{30}$ | $k_{31}$ | $k_{32}$ | $k_{33}$ |

- 下表给出了  $N_r$  与  $N_b$  和  $N_k$  的关系：

| $N_r$     | $N_b = 4$ | $N_b = 6$ | $N_b = 8$ |
|-----------|-----------|-----------|-----------|
| $N_k = 4$ | 10        | 12        | 14        |
| $N_k = 6$ | 12        | 12        | 14        |
| $N_k = 8$ | 14        | 14        | 14        |

## □ 轮函数

- AES 的轮函数由 4 个不同的计算部件组成，分别是字节代换 (ByteSub)、行移位 (ShiftRow)、列混合 (MixColumn)、密钥加 (AddRoundKey)。
- 最后一轮的轮函数与前面各轮不同，将 MixColumn 这一步去掉。



## □ 字节代换

- 字节代换(ByteSub)是一个非线性变换，独立地在每个状态字节上进行运算。它包括两个变换。
  - 在有限域  $GF(2^8)$  上求乘法逆，00 映射到它自身。
  - 在  $GF(2)$  上进行下面的仿射变换：

$$\begin{bmatrix} y_7 \\ y_6 \\ y_5 \\ y_4 \\ y_3 \\ y_2 \\ y_1 \\ y_0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} x_7 \\ x_6 \\ x_5 \\ x_4 \\ x_3 \\ x_2 \\ x_1 \\ y_0 \end{bmatrix}^{-1} + \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$

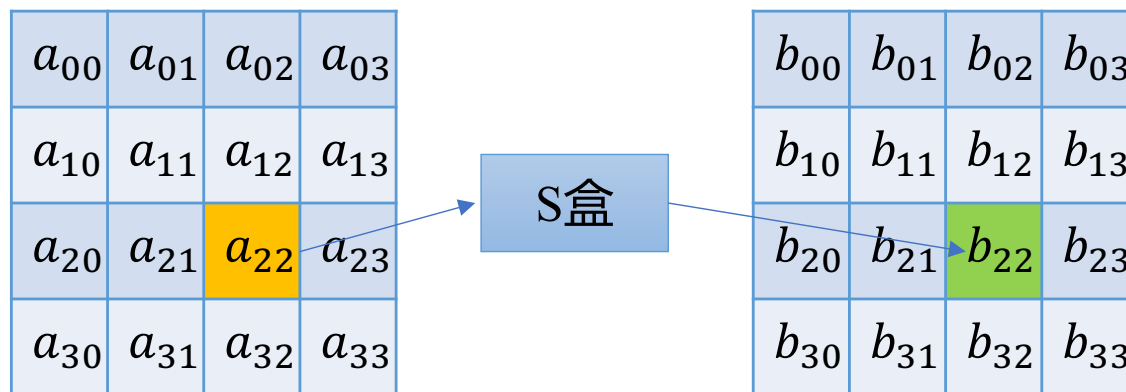
## □ 字节代换

- 可预先将  $GF(2^8)$  上的每个元素做字节代换形成代换表，也称 **S 盒**。

|   | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | A  | B  | C  | D  | E  | F  |
|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0 | 63 | 7C | 77 | 7B | F2 | 6B | 6F | C5 | 30 | 01 | 67 | 2B | FE | D7 | AB | 76 |
| 1 | CA | 82 | C9 | 7D | FA | 59 | 47 | F0 | AD | D4 | A2 | AF | 9C | A4 | 72 | C0 |
| 2 | B7 | FD | 93 | 26 | 36 | 3F | F7 | CC | 34 | A5 | E5 | F1 | 71 | D8 | 31 | 15 |
| 3 | 04 | C7 | 23 | C3 | 18 | 96 | 05 | 9A | 07 | 12 | 80 | E2 | EB | 27 | B2 | 75 |
| 4 | 09 | 83 | 2C | 1A | 1B | 6E | 5A | A0 | 52 | 3B | D6 | B3 | 29 | E3 | 2F | 84 |
| 5 | 53 | D1 | 00 | ED | 20 | FC | B1 | 5B | 6A | CB | BE | 39 | 4A | 4C | 58 | CF |
| 6 | D0 | EF | AA | FB | 43 | 4D | 33 | 85 | 45 | F9 | 02 | 7F | 50 | 3C | 9F | A8 |
| 7 | 51 | A3 | 40 | 8F | 92 | 9D | 38 | F5 | BC | B6 | DA | 21 | 10 | FF | F3 | D2 |
| 8 | CD | 0C | 13 | EC | 5F | 97 | 44 | 17 | C4 | A7 | 7E | 3D | 64 | 5D | 19 | 73 |
| 9 | 60 | 81 | 4F | DC | 22 | 2A | 90 | 88 | 46 | EE | B8 | 14 | DE | 5E | 0B | DB |
| A | E0 | 32 | 3A | 0A | 49 | 06 | 24 | 5C | C2 | D3 | AC | 62 | 91 | 95 | E4 | 79 |
| B | E7 | C8 | 37 | 6D | 8D | D5 | 4E | A9 | 6C | 56 | F4 | EA | 65 | 7A | AE | 08 |
| C | BA | 78 | 25 | 2E | 1C | A6 | B4 | C6 | E8 | DD | 74 | 1F | 4B | BD | 8B | 8A |
| D | 70 | 3E | B5 | 66 | 48 | 03 | F6 | 0E | 61 | 35 | 57 | B9 | 86 | C1 | 1D | 9E |
| E | E1 | F8 | 98 | 11 | 69 | D9 | 8E | 94 | 9B | 1E | 87 | E9 | CE | 55 | 28 | DF |
| F | 8C | A1 | 89 | 0D | BF | E6 | 42 | 68 | 41 | 99 | 2D | 0F | B0 | 54 | BB | 16 |

## □ 字节代换

- 进行字节代换时只需要查表操作：如果状态中的一个字节为  $xy$  ( $x, y$  均为4 bit)，则在 S 盒的第  $x$  行，第  $y$  列对应的字节为输出。



输入

|    |    |    |    |
|----|----|----|----|
| 12 | 2A | 21 | 0B |
| 45 | BD | 04 | C1 |
| 23 | 0A | 00 | 1C |
| 89 | 11 | 2A | FC |

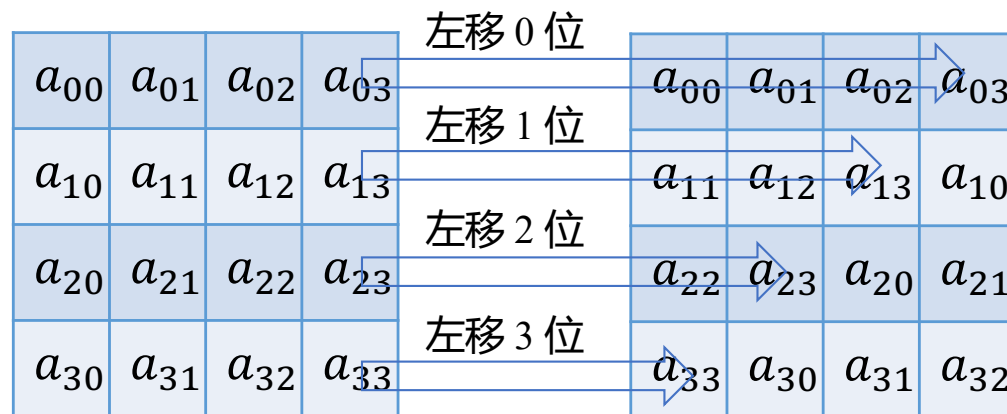
输出

|    |    |    |    |
|----|----|----|----|
| C9 | E5 | FD | 2B |
| 6E | 7A | F2 | 78 |
| 26 | 67 | 63 | 9C |
| A7 | 82 | E5 | B0 |

## □ 行移位

- 行移位 (ShiftRow) 是将状态阵列的各行进行循环移位，不同状态行的位移量不同。第 0 行不移动，第 1 行循环左移  $C_1$  个字节，第 2 行循环左移  $C_2$  个字节，第 3 行循环左移  $C_3$  个字节
- 偏移量  $C_1$ 、 $C_2$ 、 $C_3$  与分组长度  $N_b$  有关，如下表（左）所示。当  $N_b = 4$  时，行移位示意图如下图（右）所示。

| $N_b$ | $C_1$ | $C_2$ | $C_3$ |
|-------|-------|-------|-------|
| 4     | 1     | 2     | 3     |
| 6     | 1     | 2     | 3     |
| 8     | 1     | 3     | 4     |



## □ 列混合

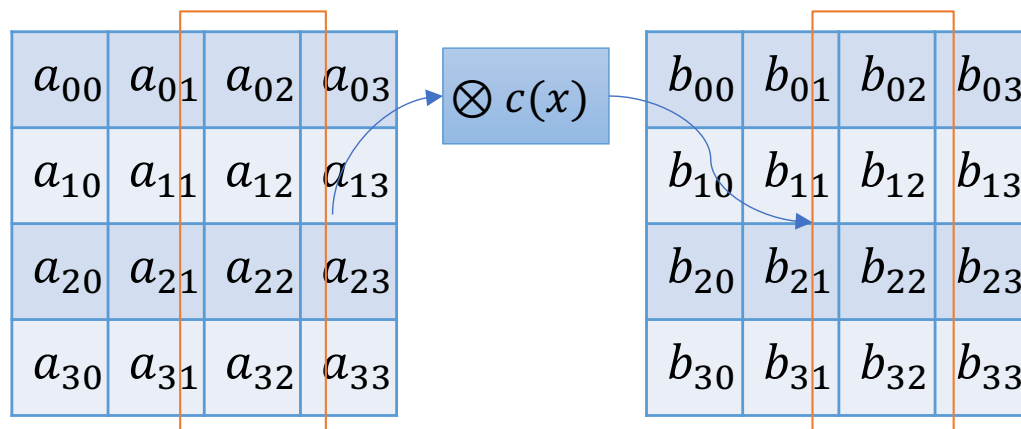
- 列混合 (MixColumn) 对一个状态逐列进行变换，即将每列视为有限域  $GF(2^8)$  上的多项式，再与一个固定的多项式  $c(x)$  进行模  $x^4 + 1$  乘法：

$$c(x) = 03x^3 + 01x^2 + 01x + 02$$

- 列混合运算也可以写为矩阵乘法。设  $b(x) = c(x) * a(x)$ ，则

$$\begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{bmatrix}$$

- 列混合运算示意图：



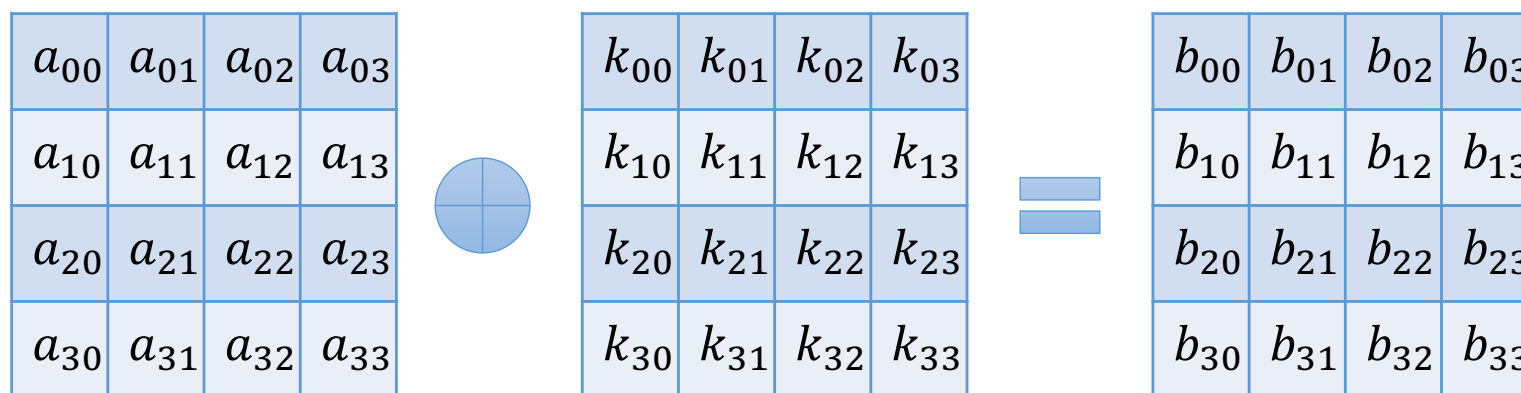


## □ 密钥加

- 密钥加(AddRoundKey)是将轮密钥简单地与状态进行逐比特异或，即：

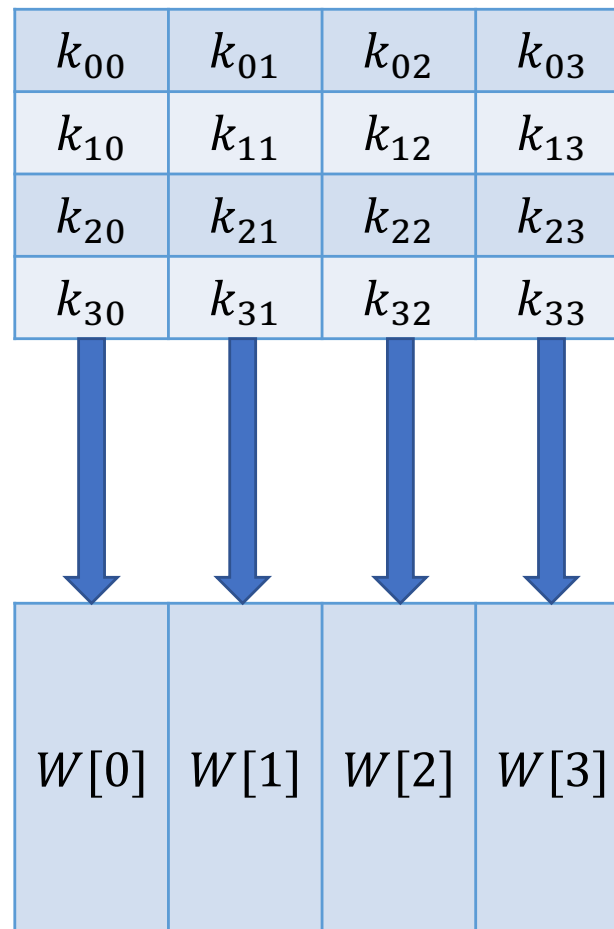
$$b_{ij} = a_{ij} \oplus k_{ij}。$$

- 密钥加运算示意图：



## □ 密钥编排

- 密钥编排是指从种子密钥得到轮密钥的过程，包括两个部分：  
密钥扩展和轮密钥选取。
- 密钥比特总数 = 分组长度  $\times$  (轮数 + 1)
  - 例如：分组长度 = 128 和 轮数 = 10 时，轮密钥长度为  $128 \times (10 + 1) = 1408$
- 首先将种子密钥输入到一个  $4 \times 4$  矩阵中，每列组成 1 个字。



### □ 密钥编排

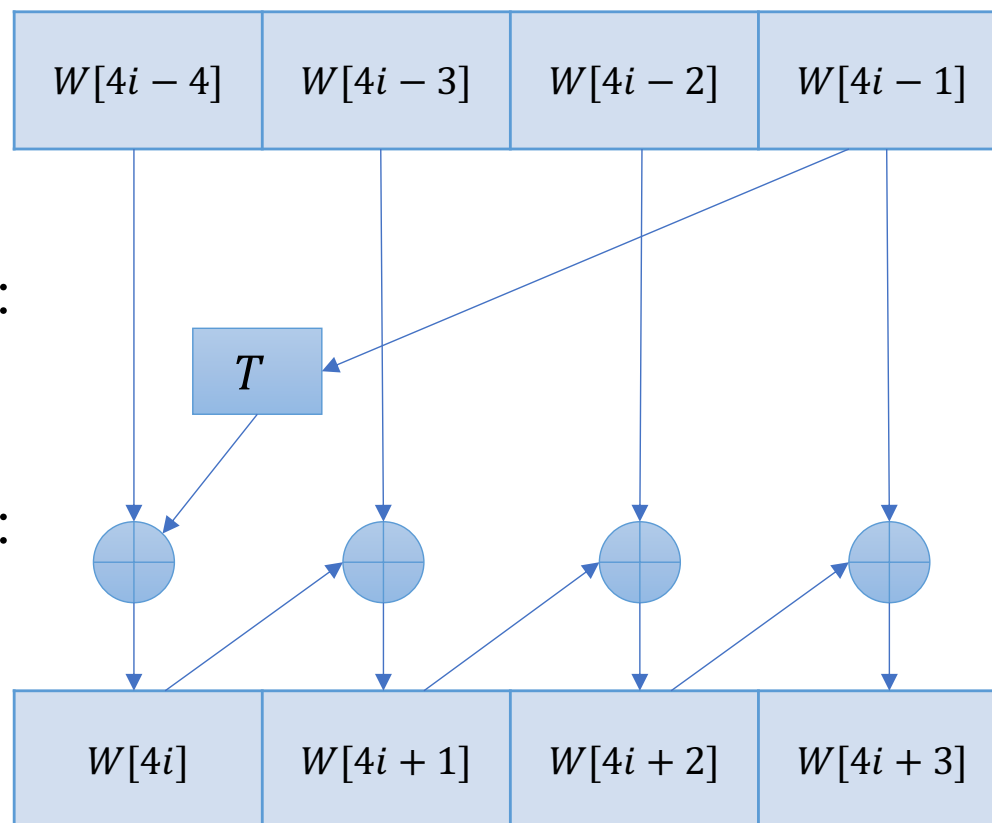
- **密钥扩展** 对  $W$  数组扩展 40 个新列，构成 44 列的密钥数组：

➤ 如果 4 不整除  $i$ ，则第  $i$  列为：

$$W[i] = W[i - 4] \oplus W[i - 1]$$

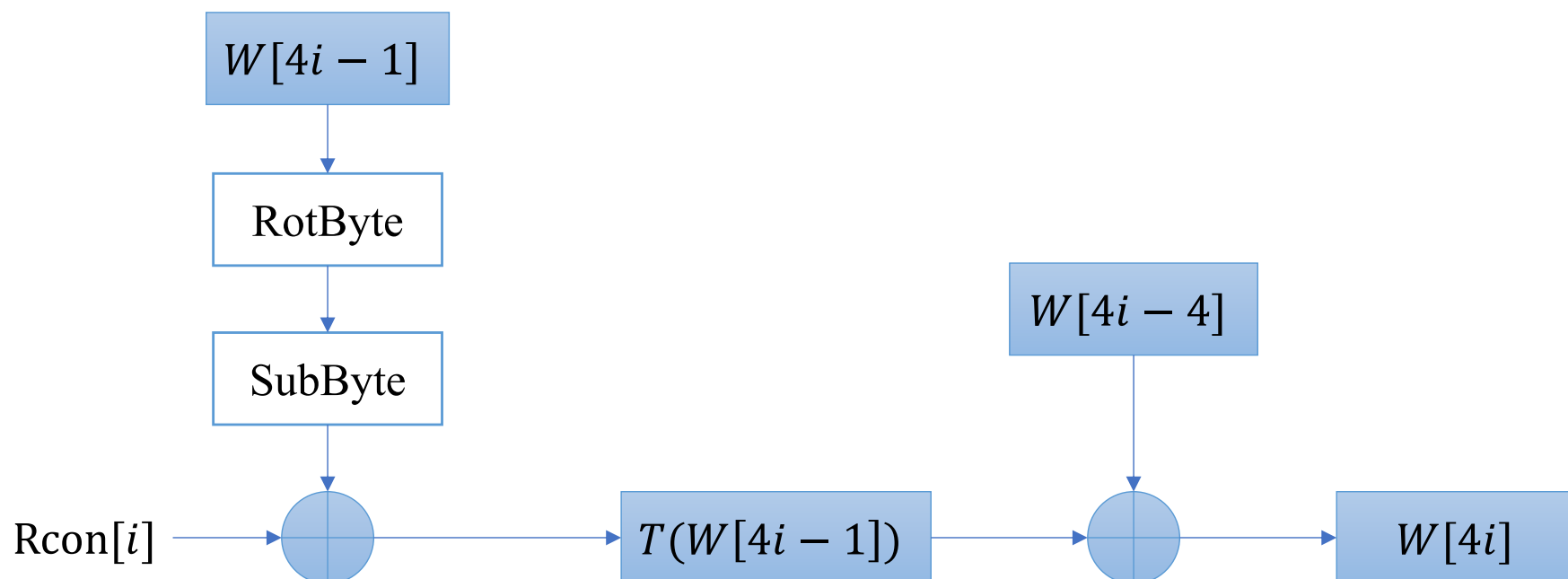
➤ 如果 4 整除  $i$ ，则第  $i$  列为：

$$W[i] = W[i - 4] \oplus T[W[i - 1]]$$



## □ 密钥编排

- $T$  函数由三部分组成：1 字节的循环移位 RotByte、用 S 盒进行变换 SubByte、异或轮常数 Rcon[i]。示意图如下：

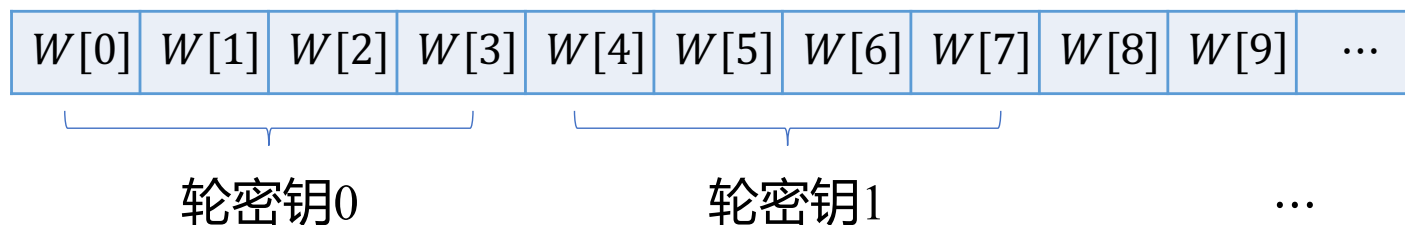


## □ 密钥编排

- 函数 RotByte 返回 4 字节字，当输入字为  $(a, b, c, d)$  时，输出字为  $(b, c, d, a)$ ，即左移 1 字节。
- 函数 SubByte 也返回 4 字节字，其中每一个字节都是用 AES 的 S 盒作用到输入字对应的字节得到。
- 轮常数 Rcon[ $i$ ] 的右边 3 个字节总是 0，记为  $Rcon[i] = (RC[i], 0, 0, 0)$ ，轮数 = 10 时的 RC[ $i$ ] 表如下：

| $i$              | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 |
|------------------|----|----|----|----|----|----|----|----|----|----|
| RC[ $i$ ] (16进制) | 01 | 02 | 04 | 08 | 10 | 20 | 40 | 80 | 1B | 36 |

- 轮密钥选取 (以  $N_k = 4$  为例)：

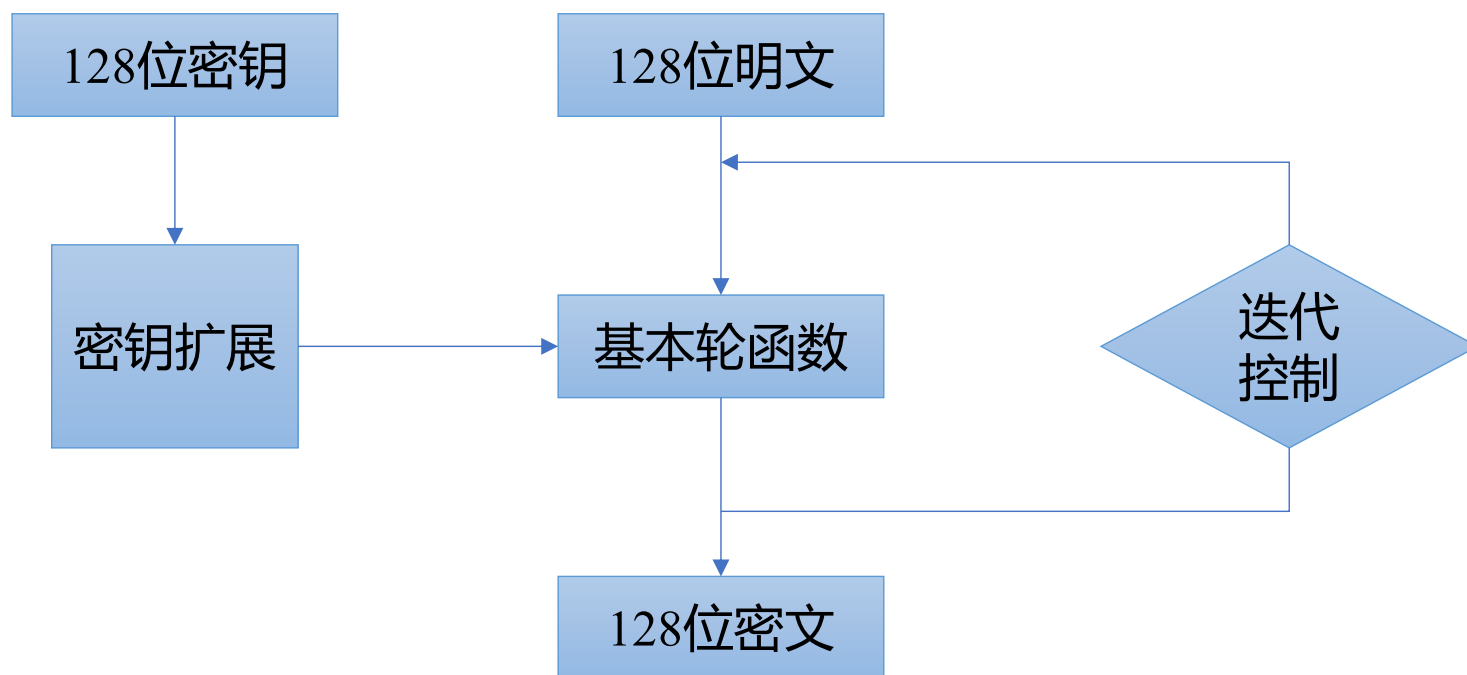


### □ SM4 概述

- 国家商用密码管理办公室制定了一系列国家密码标准，包括 SM1、SM2、SM3、SM4、SM7、SM9、祖冲之密码算法。其中 SM1、SM4、SM7、祖冲之密码是对称算法；SM2、SM9 是非对称算法；SM3 是哈希算法。
- SM4 是分组密码算法，用于 WAPI (WLAN Authentication & Privacy Infrastructure)。分组长度 128 比特，密钥长度 128 比特。加密算法与密钥扩展算法都采用 32 轮非线性迭代结构，以字节（8 位）和字节（32 位）为单位进行数据处理。
- SM4 密码算法的基本运算有模 2 加和循环移位。
  - 模 2 加：记为 $\oplus$ ，为 32 位逐比特异或运算。
  - 循环移位： $\lll i$ ，把 32 位字循环左移  $i$  位。

### □ SM4的算法结构

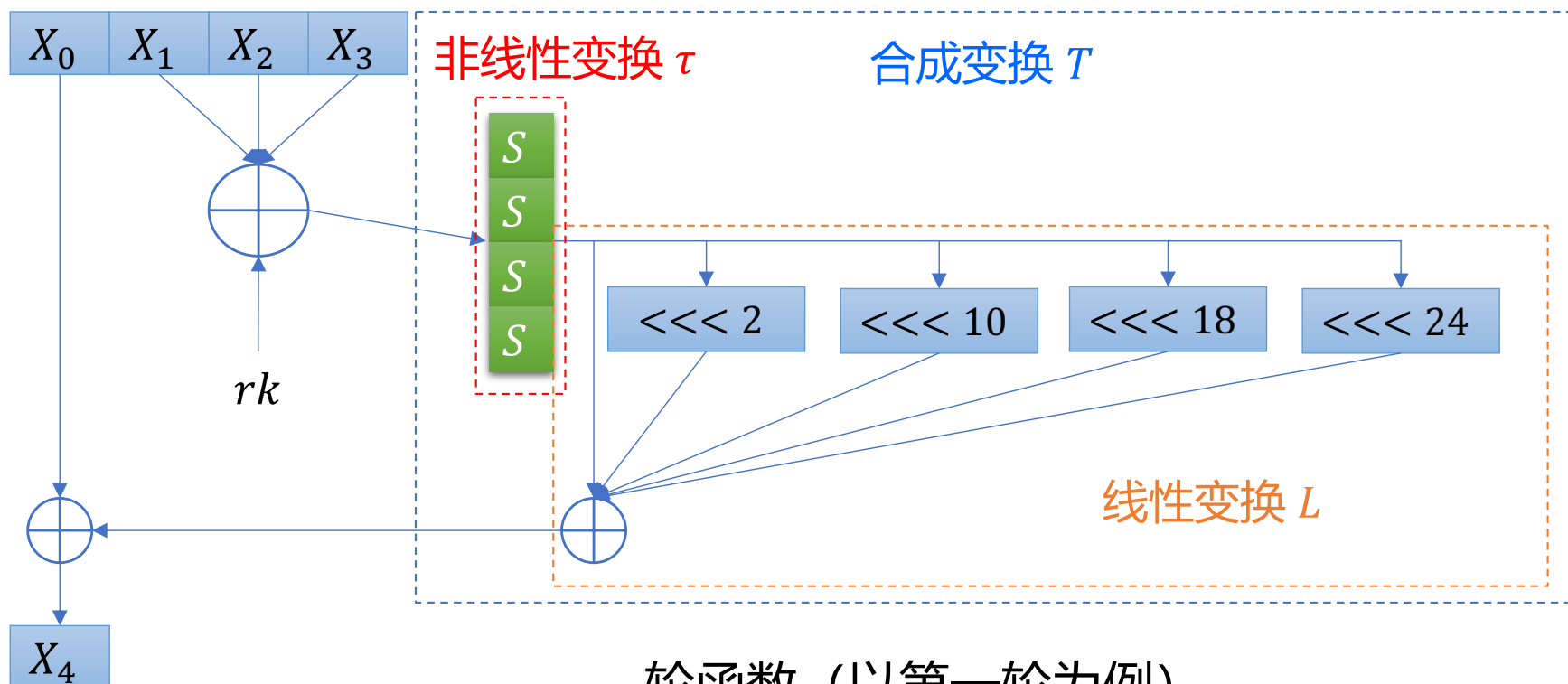
- SM4 的算法结构如下图所示：



## □ 轮函数

- 设轮函数的输入为 4 个 32 位字 ( $X_0, X_1, X_2, X_3$ ), 共128 位, 轮密钥为一个 32 位的字  $rk$ 。输出也是一个 32 位的字, 由下式给出:

$$F(X_0, X_1, X_2, X_3, rk) = X_0 \oplus T(X_1 \oplus X_2 \oplus X_3 \oplus rk)$$





## □ 非线性变换

- 非线性变换部件  $\tau$  由 4 个 S 盒并置构成，其密码学作用是混淆。每个 S 盒的输入和输出都是 8 位的字节。设输入字节为  $a$ ，输出字节为  $b$ ，则 S 盒的运算规则可表示为

$$b = S(a)$$

- 例：设输入为 “EF”，则行号为 E，列号为 F，于是 S 盒的输出值为表中第 E 行和第 F 列交叉点的值：

$$S(EF) = 84$$

## □ 非线性变换

|   | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | A  | B  | C  | D  | E  | F  |
|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0 | D6 | 90 | E9 | FE | CC | E1 | 3D | B7 | 16 | B6 | 14 | C2 | 28 | FB | 2C | 05 |
| 1 | 2B | 67 | 9A | 76 | 2A | BE | 04 | C3 | AA | 44 | 13 | 26 | 49 | 86 | 06 | 99 |
| 2 | 9C | 42 | 50 | F4 | 91 | EF | 98 | 7A | 33 | 54 | 0B | 43 | ED | CF | AC | 62 |
| 3 | E4 | B3 | 1C | A9 | C9 | 08 | E8 | 95 | 80 | DF | 94 | FA | 75 | 8F | 3F | A6 |
| 4 | 47 | 07 | A7 | FC | F3 | 73 | 17 | BA | 83 | 59 | 3C | 19 | E6 | 85 | 4F | A8 |
| 5 | 68 | 6B | 81 | B2 | 71 | 64 | DA | 8B | F8 | EB | 0F | 4B | 70 | 56 | 9D | 35 |
| 6 | 1E | 24 | 0E | 5E | 63 | 58 | D1 | A2 | 25 | 22 | 7C | 3B | 01 | 21 | 78 | 87 |
| 7 | D4 | 00 | 46 | 57 | 9F | D3 | 27 | 52 | 4C | 36 | 02 | E7 | A0 | C4 | C8 | 9E |
| 8 | EA | BF | 8A | D2 | 40 | C7 | 38 | B5 | A3 | F7 | F2 | CE | F9 | 61 | 15 | A1 |
| 9 | E0 | AE | 5D | A4 | 9B | 34 | 1A | 55 | AD | 93 | 32 | 30 | F5 | 8C | B1 | E3 |
| A | 1D | F6 | E2 | 2E | 82 | 66 | CA | 60 | C0 | 29 | 23 | AB | 0D | 53 | 4E | 6F |
| B | D5 | DB | 37 | 45 | DE | FD | 8E | 2F | 03 | FF | 6A | 72 | 6D | 6C | 5B | 51 |
| C | 8D | 1B | AF | 92 | BB | DD | BC | 7F | 11 | D9 | 5C | 41 | 1F | 10 | 5A | D8 |
| D | 0A | C1 | 31 | 88 | A5 | CD | 7B | BD | 2D | 74 | D0 | 12 | B8 | E5 | B4 | B0 |
| E | 89 | 69 | 97 | 4A | 0C | 96 | 77 | 7E | 65 | B9 | F1 | 09 | C5 | 6E | C6 | 84 |
| F | 18 | F0 | 7D | EC | 3A | DC | 4D | 20 | 79 | EE | 5F | 3E | D7 | CB | 39 | 48 |

## □ 线性变换

- 线性变换部件  $L$  是以字为处理单位的线性变换，其输入输出都是 32 位的字，它的密码学作用是扩散。
- 设  $L$  的输入为字  $B$ ，输出为字  $C$ ，则

$$\begin{aligned} C &= L(B) \\ &= B \oplus (B \lll 2) \oplus (B \lll 10) \oplus (B \lll 18) \oplus (B \lll 24) \end{aligned}$$

合成变换  $T$  由非线性变换  $\tau$  和线性变换  $L$  复合而成，数据处理单位是字。设输入为字  $X$ ，则先对  $X$  进行非线性  $\tau$  变换，再进行线性  $L$  变换。记为

$$T(X) = L(\tau(X))$$

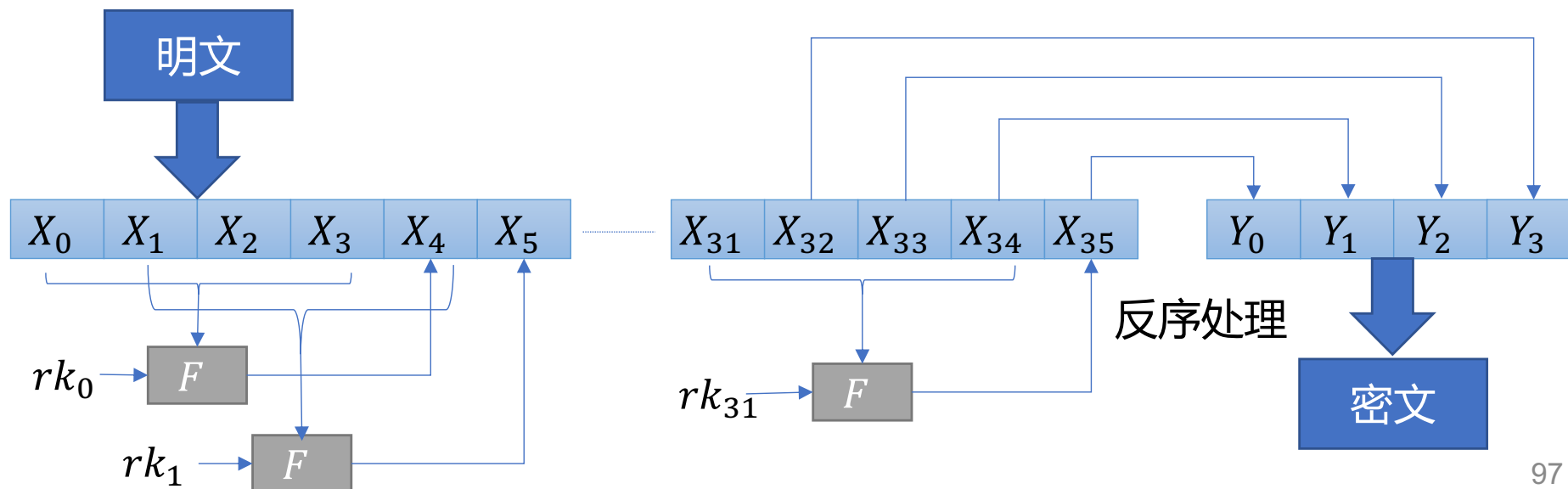
## □ 加解密算法

- 设输入明文为  $(X_0, X_1, X_2, X_3)$ ，输入的轮密钥为  $rk_i (i = 0, 1, \dots, 31)$ 。则输出的密文为  $(Y_0, Y_1, Y_2, Y_3)$ 。加密算法描述如下：

For  $i = 0, 1, \dots, 31$  Do  $X_{i+4} = F(X_i, X_{i+1}, X_{i+2}, X_{i+3}, rk_i)$

$$(Y_0, Y_1, Y_2, Y_3) = (X_{35}, X_{34}, X_{33}, X_{32})$$

- 为了与解密算法需要的顺序一致，在加密算法后还需要一个反序处理。解密算法与加密算法相同，只是轮密钥的使用顺序相反。



## □ 密钥扩展算法

- 设输入的加密种子密钥为  $MK = (MK_0, MK_1, MK_2, MK_3)$ , 输出轮密钥为  $rk_i (i = 0, 1, \dots, 30, 31)$ , 密钥扩展算法描述如下:

$$\text{➤ } (K_0, K_1, K_2, K_3) = (MK_0 \oplus FK_0, MK_1 \oplus FK_1, MK_2 \oplus FK_2, MK_3 \oplus FK_3)$$

➤ For  $i = 0, 1, \dots, 31$  Do

$$rk_i = K_{i+4} = K_i \oplus T'(K_{i+1} \oplus K_{i+2} \oplus K_{i+3} \oplus CK_i)$$

其中的变换  $T'$  与加密算法轮函数中的  $T$  基本相同, 只将其中的线性变化  $L$  修改为以下的  $L'$ :

$$L'(B) = B \oplus (B \lll 13) \oplus (B \lll 23)$$

## □ 密钥扩展算法：常数 FK 和固定参数 CK

- 在密钥扩展算法中使用如下的常数：

$$FK_0 = (A3B1BAC6), FK_1 = (56AA3350),$$

$$FK_2 = (677D9197), FK_3 = (B27022DC)$$

- 共使用 32 个固定参数  $CK_i$ ，每个  $CK_i$  是一个字，其产生规则如下：

设  $ck_{i,j}$  为  $CK_i$  的第  $j$  个字节 ( $i = 0, 1, \dots, 31; j = 0, 1, 2, 3$ )，即  $CK_i = (ck_{i,0}, ck_{i,1}, ck_{i,2}, ck_{i,3})$ ，则  $ck_{i,j} = (4i + j) \times 7 \pmod{256}$ 。

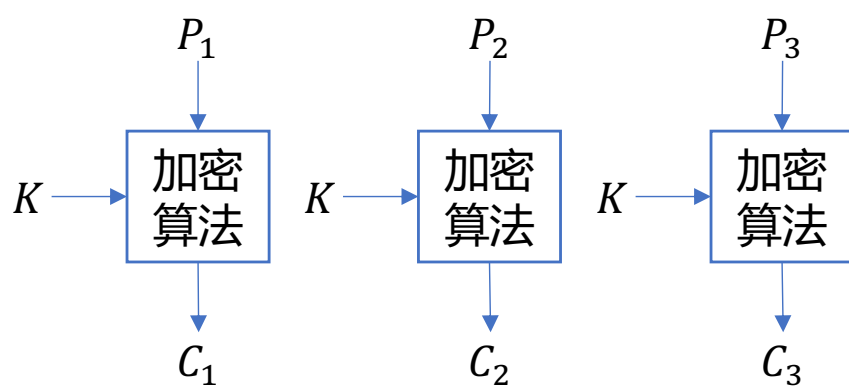
|          |          |          |          |
|----------|----------|----------|----------|
| 00070E15 | 1C232A31 | 383F464D | 545B6269 |
| 70777E85 | 8C939AA1 | A8AFB6BD | C4CBD2D9 |
| E0E7EEF5 | FC030A11 | 181F262D | 343B4249 |
| 50575E65 | 6C737A81 | 888F969D | A4ABB2B9 |
| C0C7CED5 | DCE3EAF1 | F8FF060D | 141B2229 |
| 30373E45 | 4C535A61 | 686F767D | 848B9299 |
| A0A7AEB5 | BCC3CAD1 | D8DFE6ED | F4FB0209 |
| 10171E25 | 2C333A41 | 484F565D | 646B7279 |

## □ 分组密码的工作模式

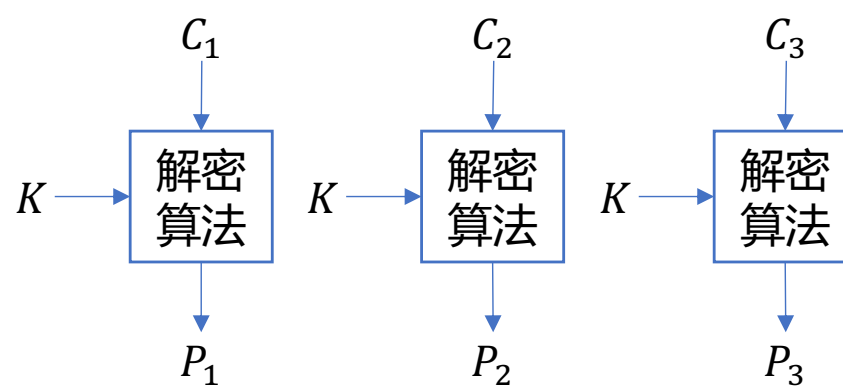
- 分组密码在加密时明文分组的长度是固定的，而实际中待加密消息的数据量是不定的，数据格式可能是多种多样的。为了能在各种应用场合使用分组密码算法，需要考虑分组密码的工作模式。另一方面，一些工作模式也可以隐蔽明文的统计特性、数据格式等，以提高整体的安全性，降低删除、重放、插入和伪造成功的机会。
- 美国在 FIPS 中定义了五种工作模式：
  - 电码本模式 (ECB)
  - 密码分组链接模式 (CBC)
  - 输出反馈模式 (OFB)
  - 密码反馈模式 (CFB)
  - 计数器模式 (CTR)

## □ 电码本模式 (ECB)

- ECB 模式直接用分组密码算法来进行消息的加密和解密。
  - 一个明文分组被加密成一个密文分组，相同的明文分组加密成相同的密文分组。明文分成 64 bit（根据算法确定）的分组进行加密，必要时填充。
  - Padding（填充）：加密消息按 64 bit 分组时，最后一组消息长度可能不足 64 bit 时可以填充一些字符。



加密



解密



### □ 电码本模式 (ECB)

#### ECB模式的优、缺点

- 优点:
  - (1) 实现简单;
  - (2) 不同明文分组的加密可并行实施, 尤其是硬件实现时速度很快
- 缺点:
  - (1) 相同明文分组对应相同密文分组
  - (2) 不能隐蔽明文分组的统计规律和结构规律, 不能抵抗替换攻击
- 应用:
  - (1) 用于随机数的加密保护
  - (2) 用于单分组明文的加密

## □ 电码本模式 (ECB)

### 电码本模式缺陷的例子

例: 假设银行A和银行B之间的资金转帐系统所使用报文模式如下:

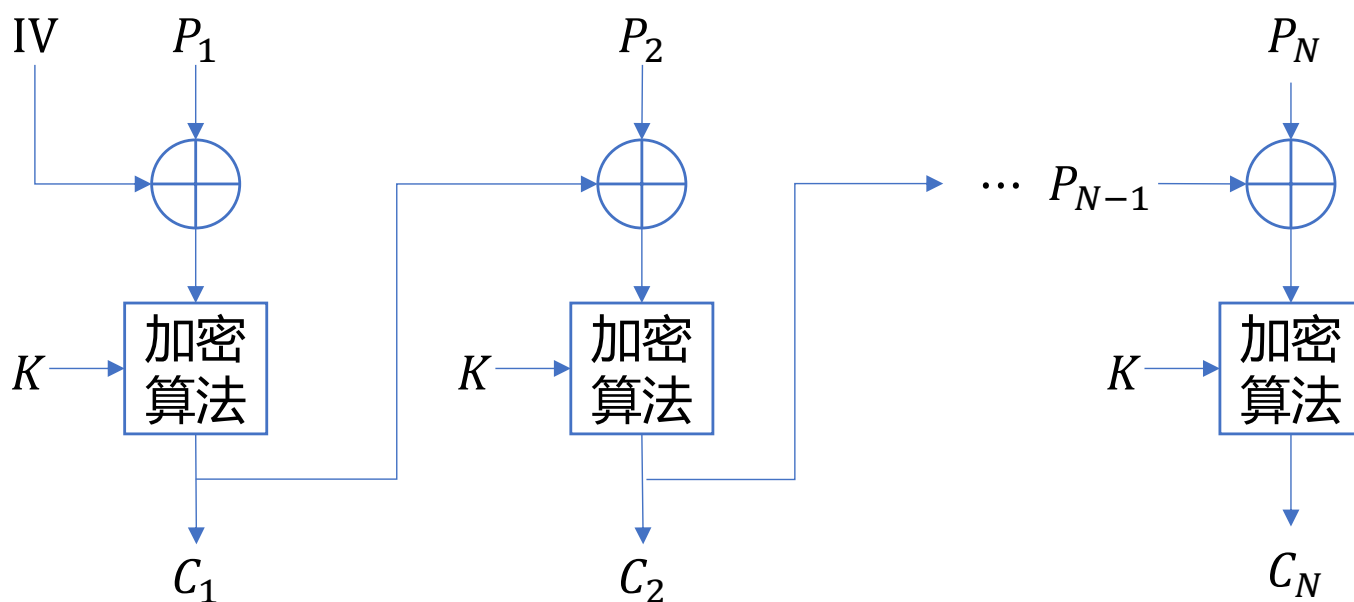
|          |          |          |        |   |   |   |   |   |    |        |          |    |
|----------|----------|----------|--------|---|---|---|---|---|----|--------|----------|----|
| 1        | 2        | 3        | 4      | 5 | 6 | 7 | 8 | 9 | 10 | 11     | 12       | 13 |
| 时间<br>标记 | 发送<br>银行 | 接收<br>银行 | 储户姓名 1 |   |   |   |   |   |    | 储户帐号 1 | 存款<br>金额 |    |

|          |          |          |        |   |   |   |   |   |    |        |    |          |
|----------|----------|----------|--------|---|---|---|---|---|----|--------|----|----------|
| 1        | 2        | 3        | 4      | 5 | 6 | 7 | 8 | 9 | 10 | 11     | 12 | 13       |
| 时间<br>标记 | 发送<br>银行 | 接收<br>银行 | 储户姓名 2 |   |   |   |   |   |    | 储户帐号 2 |    | 存款<br>金额 |

敌手C通过截收从A到B的加密消息, 只要将第5至第12分组替换为自己的姓名和帐号相对应的密文, 即可将别人的存款存入自己的帐号。

## □ 密码分组链接模式 (CBC)

- CBC 模式中，加密算法的输入是当前明文分组和前一密文分组的异或。加密算法的输入不会显示出与这次明文分组之间的固定关系，所以重复的明文分组不会在密文中暴露出这种重复关系。
- 最开始时，需要有一个初始向量 IV 与第一个明文分组异或。

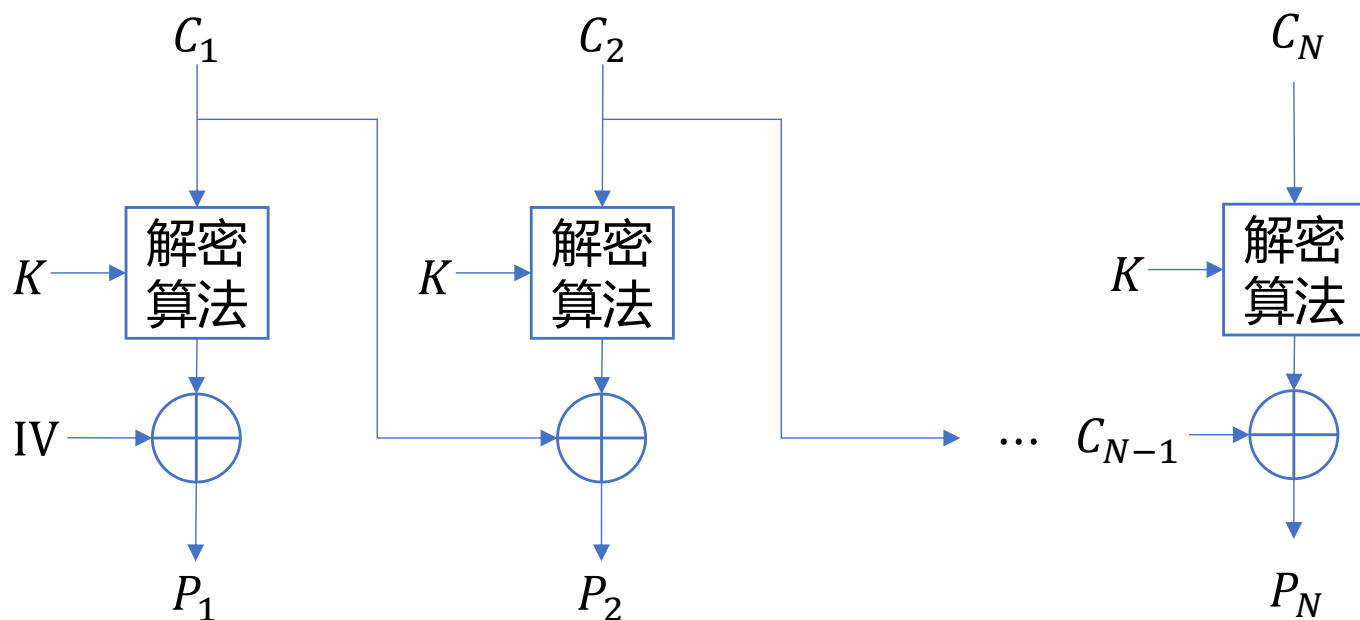


$$C_1 = E_k(P_1 \oplus IV)$$

$$C_i = E_k(P_i \oplus C_{i-1}), i = 2, 3, 4, \dots, N$$

## □ 密码分组链接模式 (CBC)

- CBC 模式解密时, IV 和解密算法对第一个密文分组的输出进行异或以恢复第一个明文分组。



$$P_1 = D_k(C_1) \oplus IV$$

$$P_i = D_k(C_i) \oplus C_{i-1}, i = 2, 3, 4, \dots, N$$

### □ 密码分组链接模式 (CBC)

1. 明文块的统计特性得到了隐蔽
  - 由于在CBC模式中，各密文块不仅与当前明文块有关，而且还与以前的明文块及初始化向量有关，从而使明文的统计规律在密文中得到了较好的隐藏
2. 具有有限的(两步)错误传播特性
  - 一个密文块的错误将导致两个密文块不能正确解密
3. 具有自同步功能
  - 密文出现丢块和错块不影响后续密文块的解密.若从第 $t$ 块起密文块正确,则第 $t+1$ 个明文块就能正确求出

### □ 密码分组链接模式 (CBC)

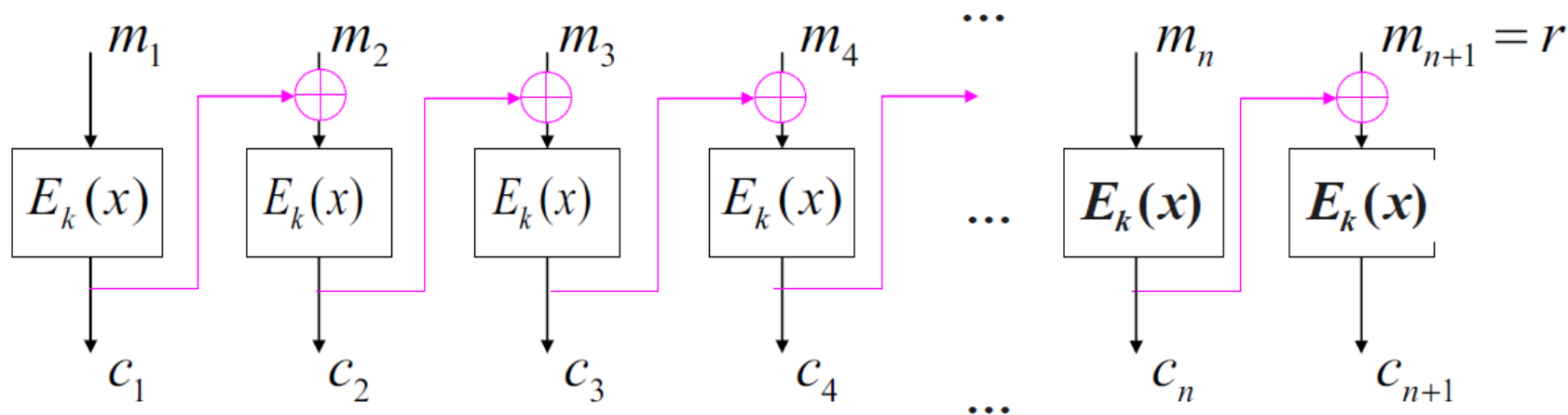
- 目的:检查文件在(直接或加密)传输和存储中是否遭到有意或无意的篡改.
- 关键技术:
  - (1) 文件的制造者和检验者共享一个密钥
  - (2) 文件的明文必须具有检验者预先知道的冗余度
  - (3) 文件的制造者用共享密钥对具有约定冗余度的明文用CBC模式加密
  - (4) 文件的检验者用共享密钥对密文解密, 并检验约定冗余度是否正确

## □ 密码分组链接模式 (CBC)

$n$ 个分组明文  $m = (m_1, \dots, m_n)$ , 校验码为

$$r = m_{n+1} = m_1 \oplus \dots \oplus m_n$$

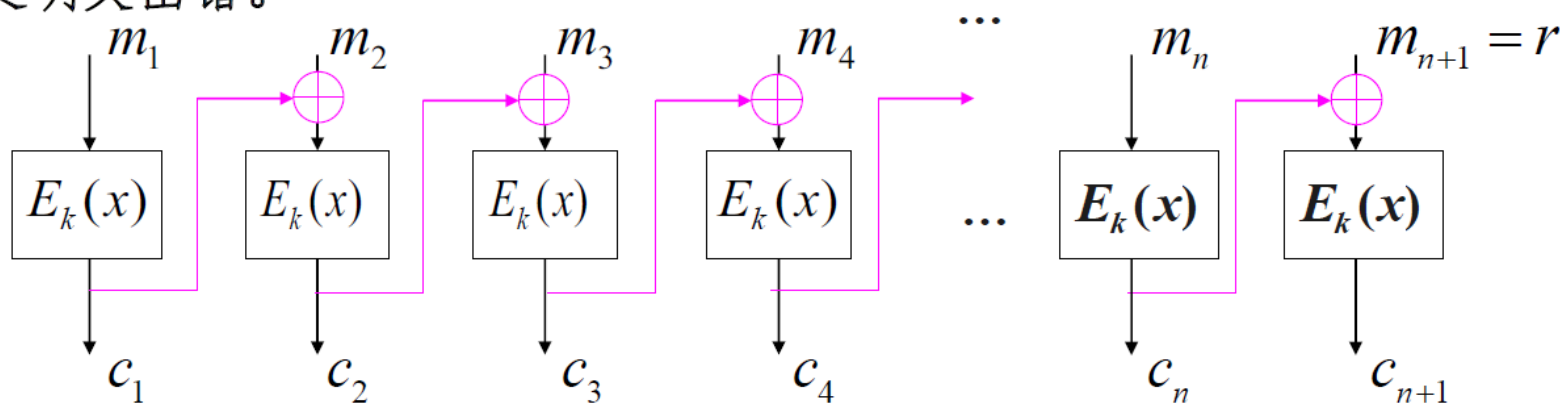
$C_{n+1}$ 为认证码。



- (1) 仅需对明文认证,而不需加密时,传送明文 $m$ 和认证码 $C_{n+1}$ , 此时也可仅保留 $C_{n+1}$ 的  $t$  个比特作为认证码;
- (2) 既需对明文认证,又需要加密时,传送密文 $C$ 和认证码 $C_{n+1}$

## □ 密码分组链接模式 (CBC)

- (1) 仅需对明文认证而不需加密时,此时验证者仅收到明文 $m$ 和认证码 $C_{n+1}$ , 他需要:
- Step1 产生明文 $m$ 的校验码  $r = m_{n+1} = m_1 \oplus \dots \oplus m_n$
- Step2 利用共享密钥使用CBC模式对 $(m,r)$ 加密, 将得到的最后一个密文分组与接收到的认证码 $C_{n+1}$ 比较, 二者一致时判定接收的明文无错; 二者不一致时判定明文出错。





### □ 密码反馈模式 (CFB)

- CFB 模式将分组密码转换为流密码。输入是 64 比特的移位寄存器，其初值为初始向量 IV。输出最左边  $j$  比特与明文第一个单元  $P_1$  进行异或，产生密文的第 1 个单元  $C_1$ 。然后将移位寄存器左移  $j$  位，并将  $C_1$  送入移位寄存器的最右边  $j$  位，直至明文所有单元被加密。

若待加密消息需按字符、字节或比特处理时，可采用CFB模式。并称待加密消息按 $r$  比特处理的CFB模式为 $r$  比特CFB模式。

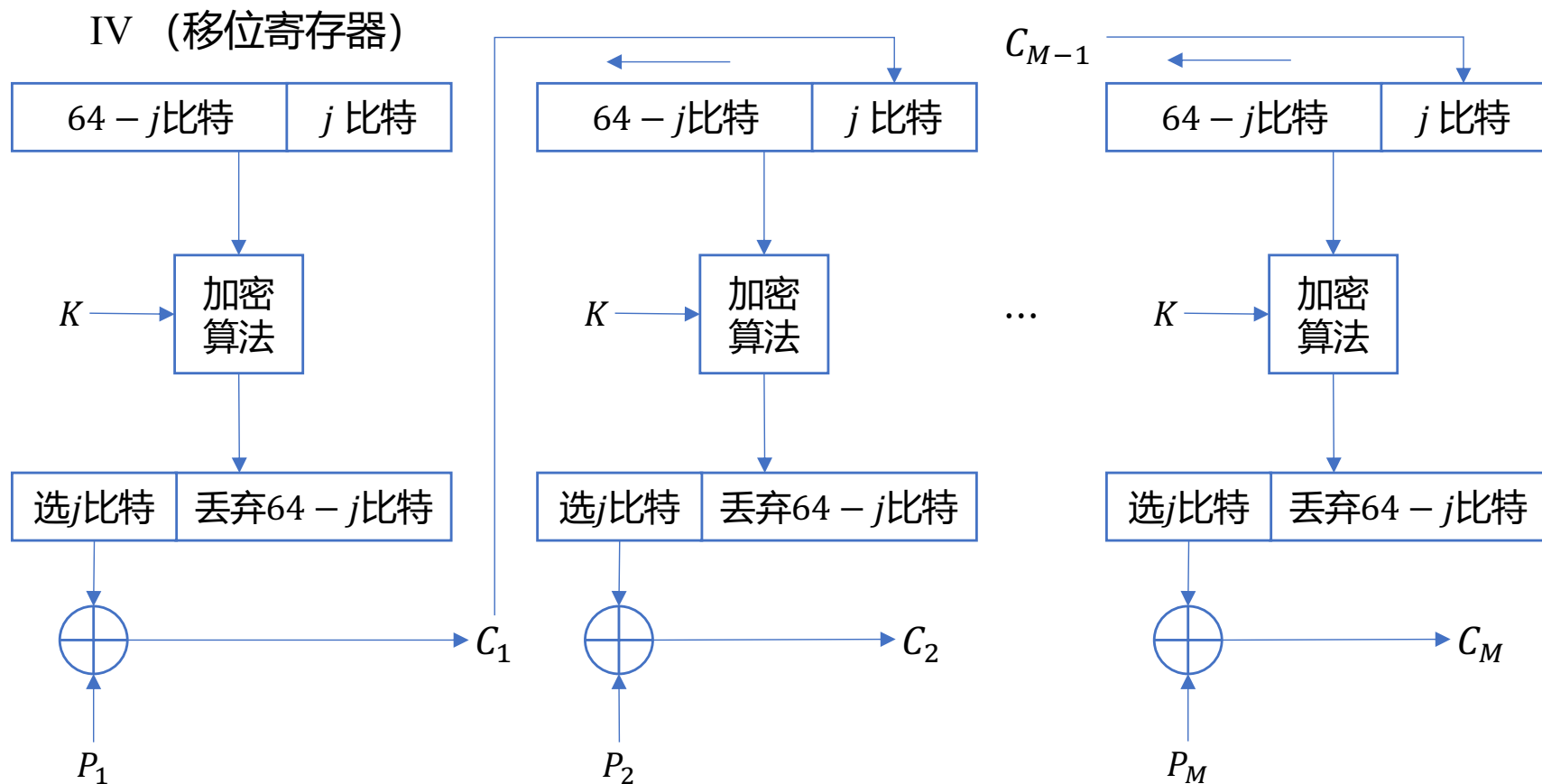
#### 适用范围:

适用于每次处理 $r$ 比特明文块的特定需求的加密情形,能灵活适应数据各格式的需要。

例如,数据库加密要求加密时不能改变明文的字节长度,这时就要以明文字节为单位进行加密。

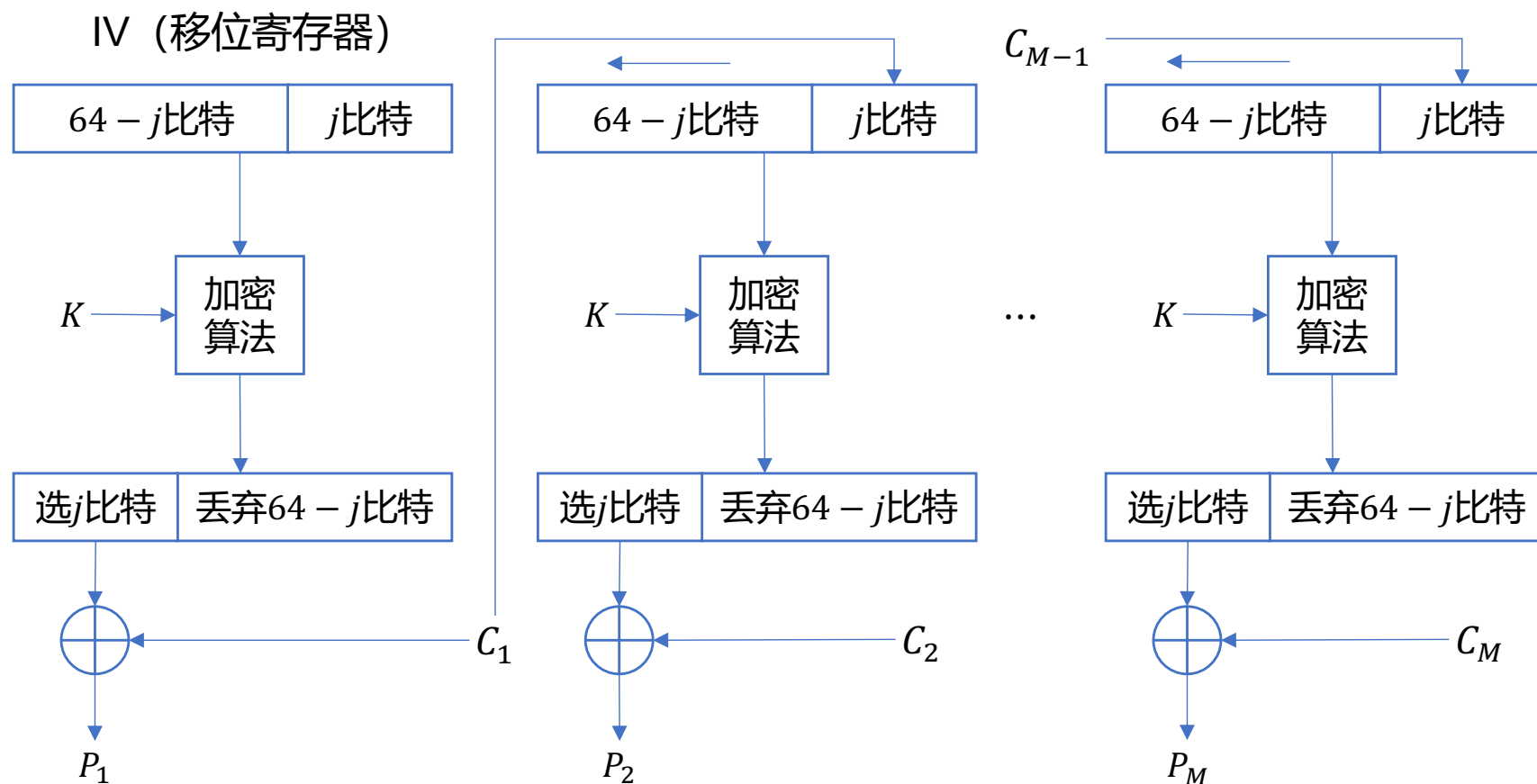
## □ 密码反馈模式 (CFB)

- CFB 模式将分组密码转换为流密码。输入是 64 比特的移位寄存器，其初值为初始向量 IV。输出最左边  $j$  比特与明文第一个单元  $P_1$  进行异或，产生密文的第 1 个单元  $C_1$ 。然后将移位寄存器左移  $j$  位，并将  $C_1$  送入移位寄存器的最右边  $j$  位，直至明文所有单元被加密。



## □ 密码反馈模式 (CFB)

- CFB 模式的解密仍使用分组密码的加密算法而不是解密算法。



### □ 密码反馈模式 (CFB)-特点

**相同明文：**和按CBC模式加密一样，改变IV同样会导致相同的明文输入得到不同的加密输出。IV无需保密（虽在某些应用中IV须是不可预测的）。

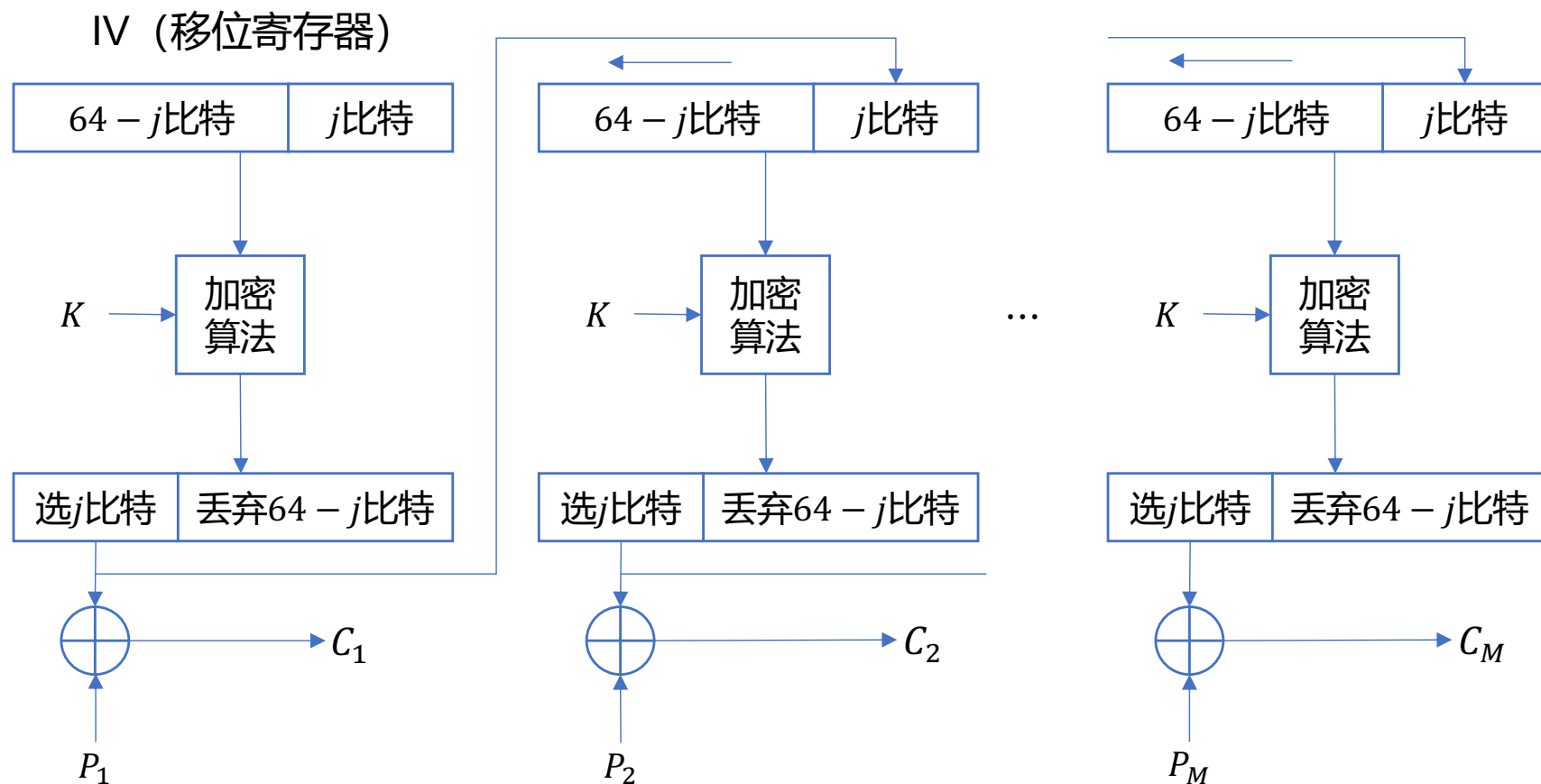
**链接依赖性：**类似CBC加密，链接机制致使密文组依赖于当前明文组和其前面的明文组；因此，**重排密文组会影响解密。**

**错误的传播：**一个或多个比特错误出现在任一个r比特的密文组中会影响这个组和后继 $\lceil n/r \rceil$ 个密文组的解密。

**错误恢复：**CFB和CBC相似，也是自同步的，但它需有  $\lceil n/r \rceil$  个个密文组才能还原。

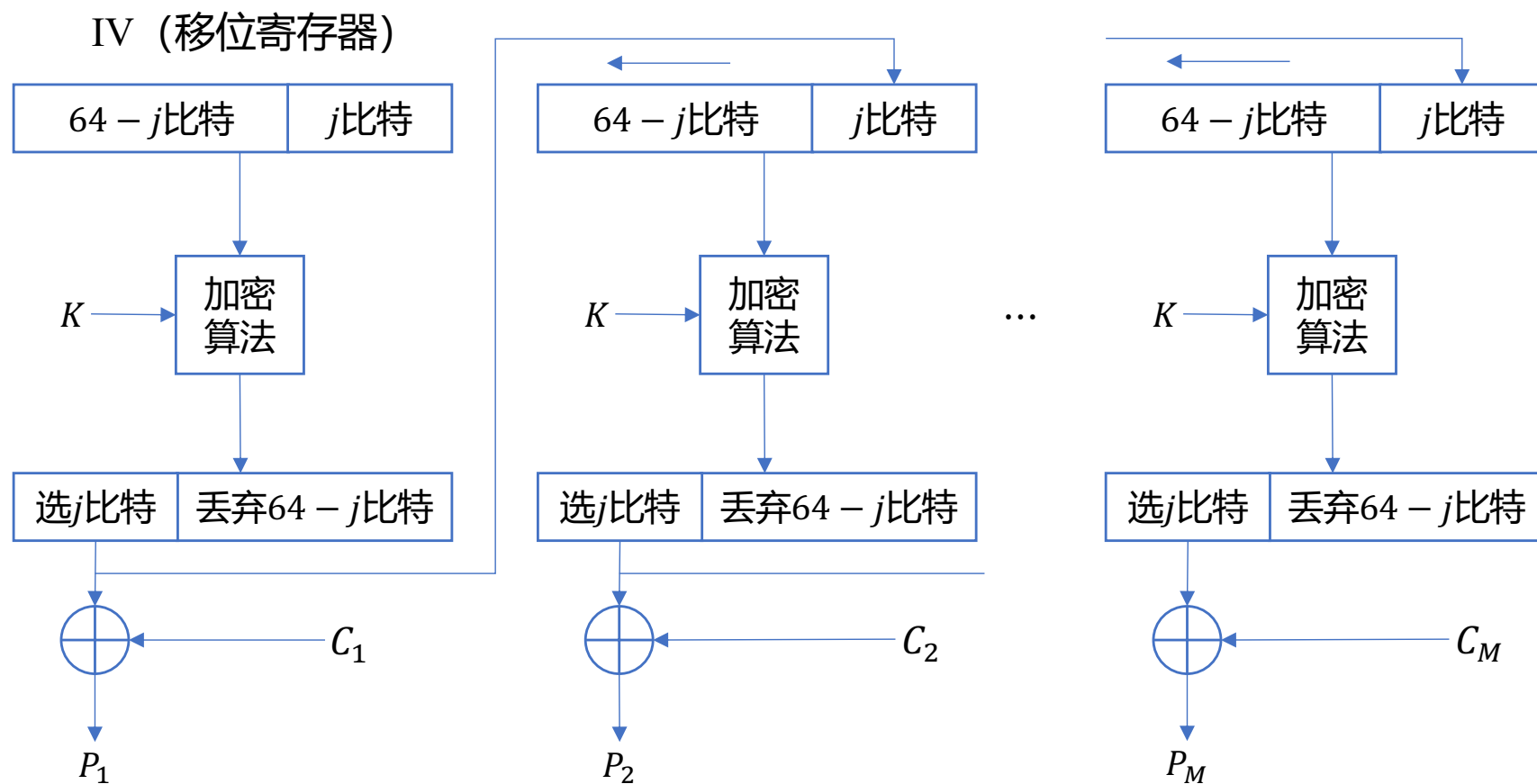
## □ 输出反馈模式 (OFB)

- OFB 模式的结构与 CFB 的结构类似，不同之处在于：OFB 模式将分组密码的加密算法的输出反馈到移位寄存器。



## □ 输出反馈模式 (OFB)

- 类似于 CFB 模式，OFB 模式的解密使用的是分组密码的加密算法。



### □ 输出反馈模式 (OFB)

**相同明文：**和CBC及CFB一样，改变 $IV$ 同样会导致相同的明文输入得到不同的加密输出。

**链接依赖性：**密钥流是独立于明文的。

**错误传播：**有一个或多个比特错误的任一密文字符仅会影响该字符的解密，密文字符的某比特位置出错将致使还原明文的相应位置也出错。

**错误恢复：**OFB模式能从密文比特错误中得以恢复，但在丢失密文比特后就无法实现自同步了，这是因为丢失密文比特会破坏密钥流的编排。

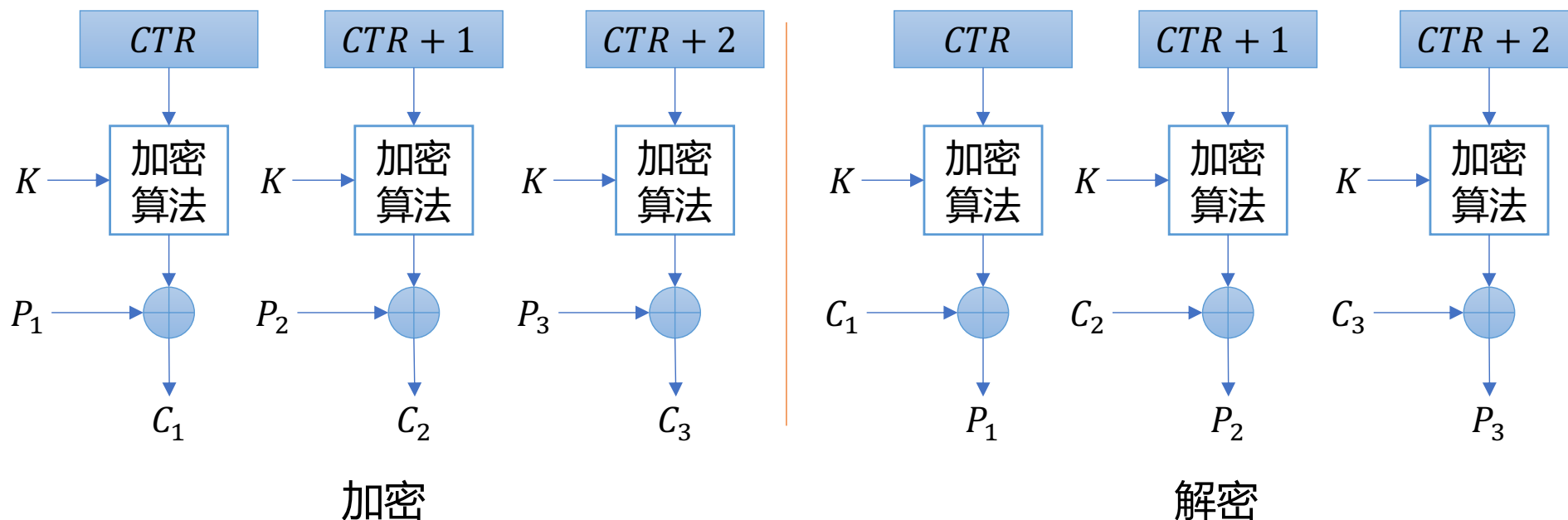
## 四类工作模式比较和选用

- ECB模式简单、高速，但最弱，易受重放和替换攻击，一般用于加密长度小于等于分组长度的消息。
- CBC, CFB, OFB模式的选用取决于实际的特殊需求。
  - 明文不易丢信号，对明文的格式没有特殊要求的环境可选用CBC模式。需要完整性认证功能时也可选用该模式。
  - 容易丢信号的环境，或对明文格式有特殊要求的环境，可选用CFB模式。
  - 不易丢信号，但信号特别容易错，且明文冗余特别多，可选用OFB模式。



## □ 计数器模式 (CTR)

- CTR 模式使用与明文分组相同的计数器长度，但加密不同的分组所用的计数器值必须不同。计数器从某一初值开始，依次递增 1。计数器值经分组密码的加密算法变换后的结果与明文分组异或，从而得到密文；解密时使用相同的计数器值序列，经加密算法变换后的计数器值与密文分组异或，从而恢复明文。



### □ 工作模式的比较和选用

- ECB 模式：简单、高速，但最弱、易受重放攻击，一般不推荐。
- CBC 适用于文件加密，但较 ECB 慢。安全性加强。
- OFB 和 CFB 较 CBC 慢许多。每次迭代只有少数比特完成加密。若可以容忍少量错误扩展，此时用 CFB。在字符为单元的流密码中多选 CFB 模式。
- OFB 用于高速同步系统，没有错误传播。
- CTR 适用于对实时性和速度要求比较高的场合。

## □ 工作模式的比较和选用

| 模式  | 描述                                                      | 特点                               | 用途                |
|-----|---------------------------------------------------------|----------------------------------|-------------------|
| ECB | 每个明文组独立地以同一密钥加密                                         | 明文相同，密文相同；操作简单，容易受到重放和代换攻击；无差错传播 | 传送短数据（如一个加密密钥）    |
| CBC | 加密算法的输入是当前明文组和前一密文组                                     | 初始向量 IV 保密和完整性，密文差错传播            | 传输数据分组；认证         |
| CTR | 加密算法的输入为当前计数器的值，该输出值与当前明文异或以产生密文                        | 只需加密，没有解密；简单，可预处理，并行处理           | 实时性和速度要求比较高的加密场合  |
| OFB | 与CFB相似，不同之处是本次加密算法的输入为前一次加密算法的输出                        | 分组密码算法作为一个密钥流产生器，对于密文被篡改难以进行检测   | 有扰信道上(如卫星通讯)传送数据流 |
| CFB | 每次只处理输入的 $j$ 比特，将上一次密文用作加密算法的输入以产生伪随机输出，该输出与当前明文异或以产生密文 | 有差错传播                            | 传送数据流，认证          |

- ❑ 私钥密码体制的特征是加密密钥和解密密钥相同，特点是加、解密速度快，密钥相对较短。私钥密码包括流密码和分组密码。
- ❑ 分组密码的加密算法通常由多轮相同或相近的轮函数组成，每一轮所用的密钥由密钥扩展算法产生；分组密码的解密算法是加密算法的逆操作。
- ❑ 分组密码的设计理念体现了 Shannon 密码学的思想：扩散与混淆。其中置换或移位的作用是扩散，非线性变换组件 S 盒的作用是混淆。
- ❑ 为了能在各种应用场合高效安全的使用分组密码算法，需要考虑分组密码的工作模式，包括电码本模式（ECB）、密码分组链接模式（CBC）、输出反馈模式（OFB）、密码反馈模式（CFB）和计数器模式（CTR）。



西安电子科技大学  
XIDIAN UNIVERSITY

西安电子科技大学

团结 勤奋

求实 创新

谢谢！

THANK YOU

